CM09 - Parameters and Main

Pass by value, pointer parameters, arrays, main arguments

Louis Ledoux

2026-07-05

Table of contents

1 Pass by Value

Live pacing: about 8 min

Question. What is the role of Pass by Value in the course story?

Core claim. Pass by Value is one step in the path from source text to reliable execution.

Target capability. Students can connect the topic to a concrete command, program state, or memory state.

1.1 Main Path

- Start from the question: What is the role of Pass by Value in the course story?
- Build the model: Pass by Value is one step in the path from source text to reliable execution.
- End with a checkable action: Students can connect the topic to a concrete command, program state, or memory state.
- Keep only this slide on the horizontal path during the live lecture.

1.2 Worked Example

- Use this as the live trace: Use one short trace, then ask students which state changed and which state did not.
- Representative source slide: 256
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

1.3 Anecdote Or Motivation

Keep the discussion anchored in observable behavior: command output, compiler message, or memory drawing.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

1.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Pass by value

- void proc(int n){
- n++;
- }
- void main{
- ...
- int a=10;
- proc(a);
- a = a + 10;
- }

- stack

- main

- a = 20

#260

1.5 Backup Index

Original slides 256-260

Passing arguments: By value

- Function parameters: LOCAL variables initialized with the value of the passed argument

- Pass by Value: pass the data object itself
- The parameter is initialized with the actual value of the data object
- When a function returns:
- The space reserved for the local variables of the function is erased!
- With that, any modification occurred on the local variables

#256

slide 256

Pass by value

- void proc(int n){
- void main{
- ...
- int a=10;
- proc(a);
- }

- stack

- n = 10
- proc

- main
- a = 10

#257

slide 257

Pass by value

- void proc(int n){
- n++;
- void main{
- ...
- int a=10;
- proc(a);
- }

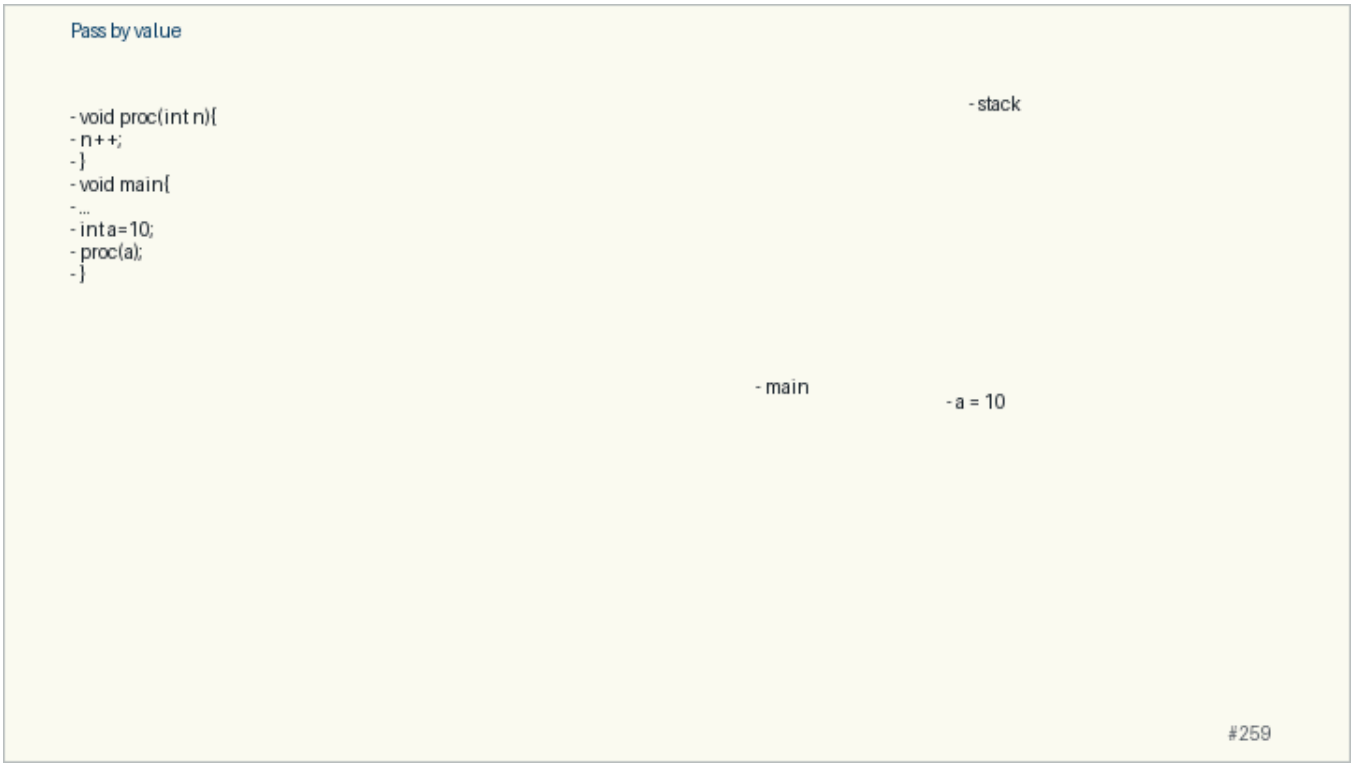
- stack

- n = 11
- proc

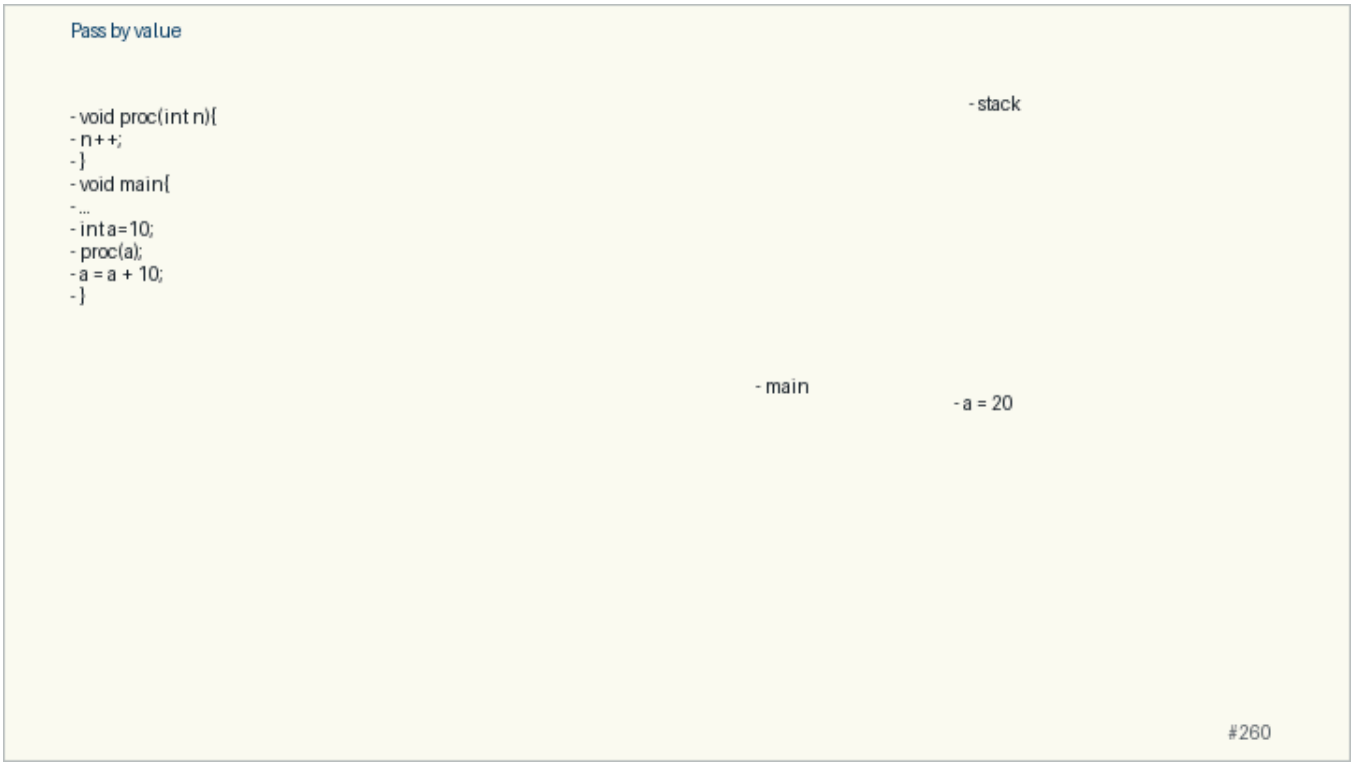
- main
- a = 10

#258

slide 258



slide 259



slide 260

Anchors: Passing arguments: By value; Pass by value

1.6 Backup: Passing arguments: By value

Original slide 256

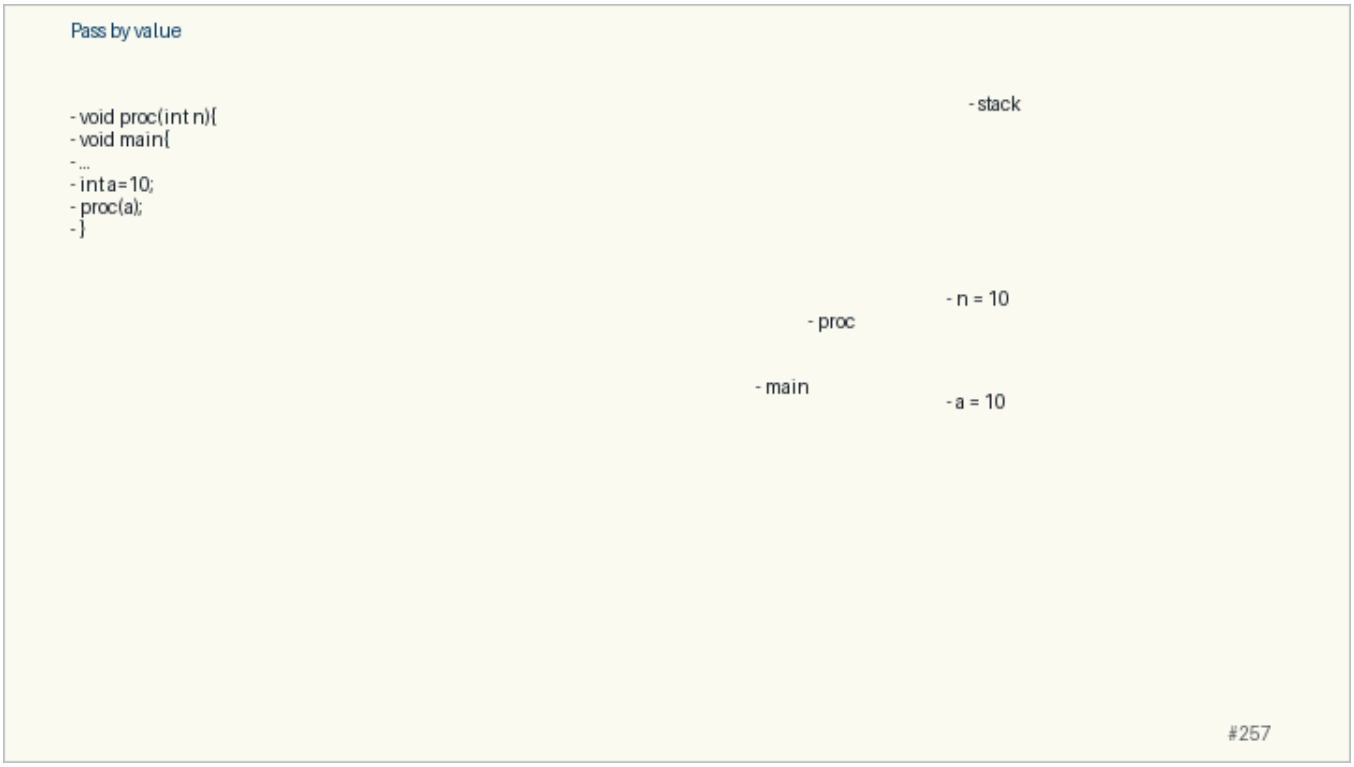
- Function parameters: LOCAL variables initialized with the value of the passed argument
- Pass by Value: pass the data object itself
- The parameter is initialized with the actual value of the data object

- When a function returns:
- The space reserved for the local variables of the function is erased!
- With that, any modification occurred on the local variables

1.7 Backup: Pass by value

Original slide 257

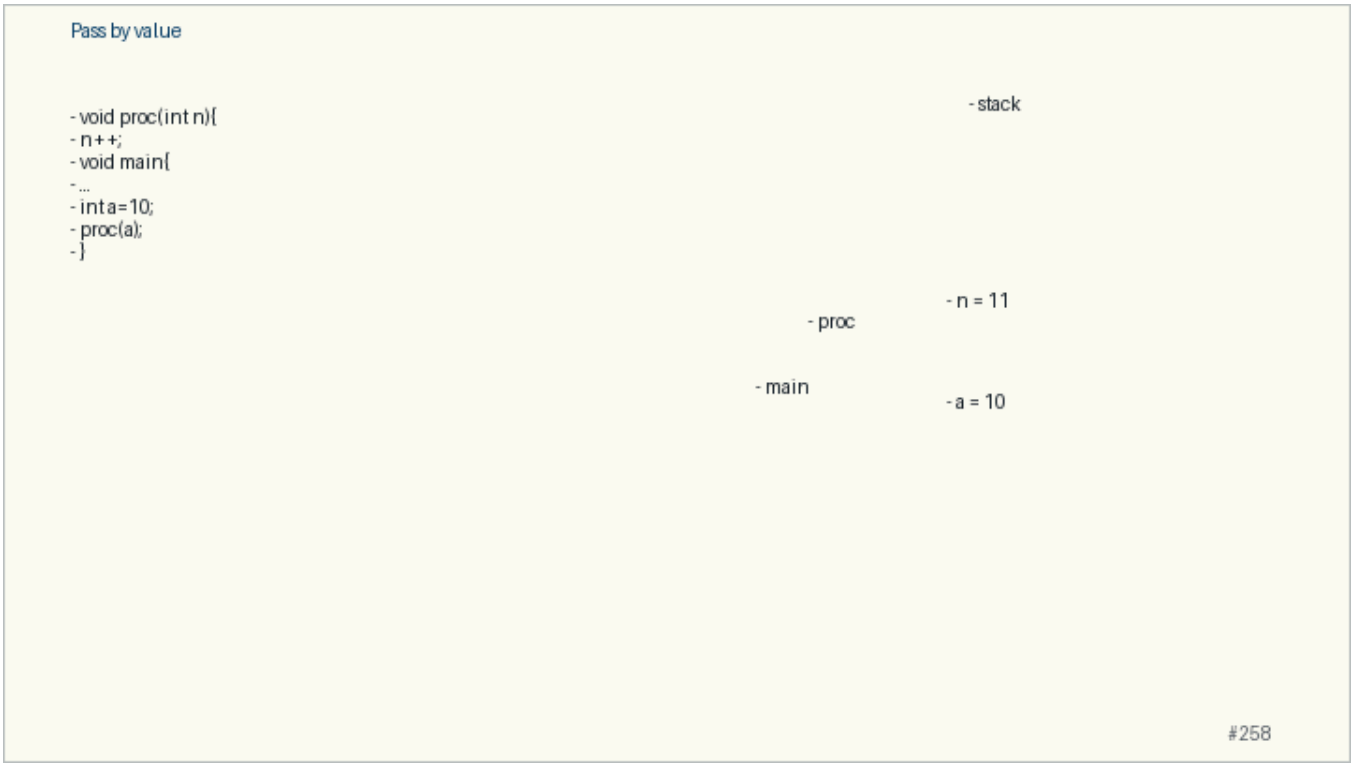
- stack
- void proc(int n){
- void main{
- ...
- int a=10;
- proc(a);
- }
- n = 10
- proc
- main
- a = 10



1.8 Backup: Pass by value

Original slide 258

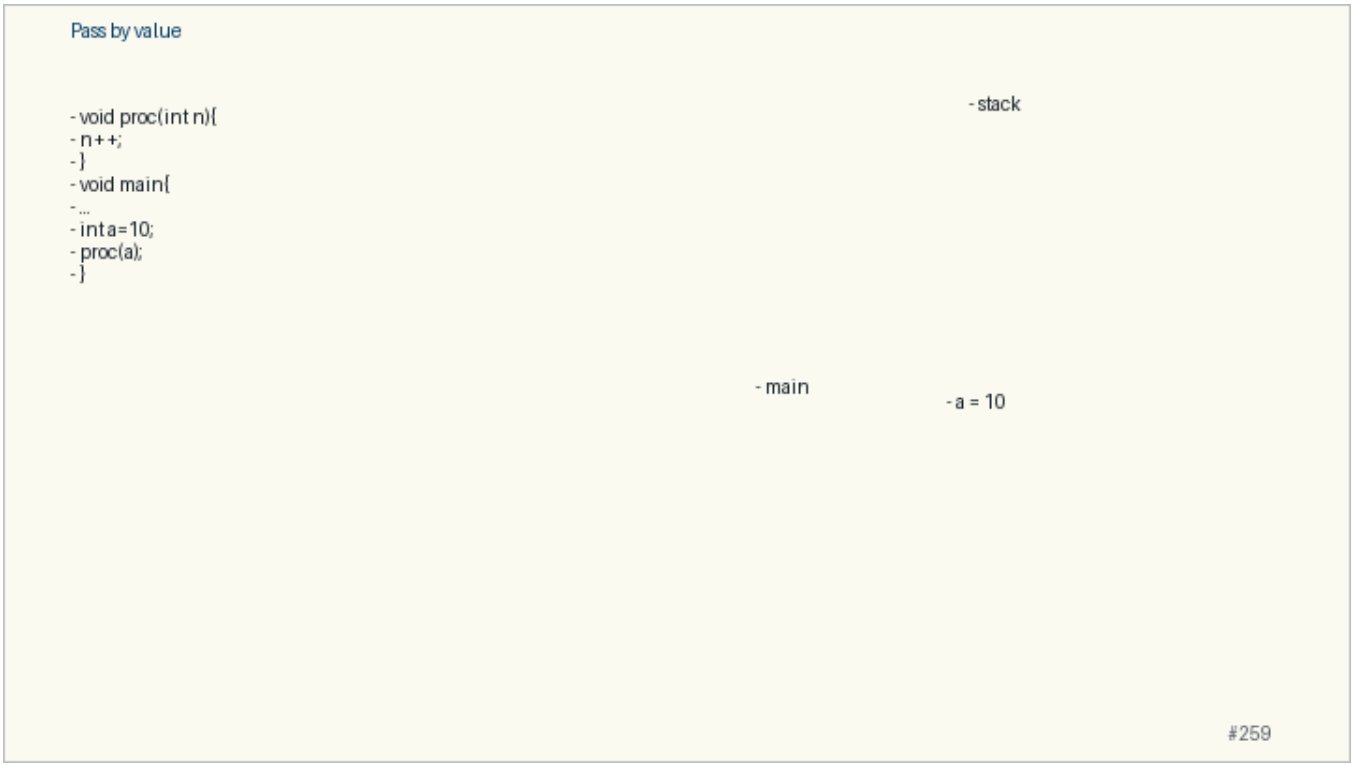
- stack
- void proc(int n){
- n++;
- void main{
- ...
- int a=10;
- proc(a);
- }
- n = 11
- proc
- main
- a = 10



1.9 Backup: Pass by value

Original slide 259

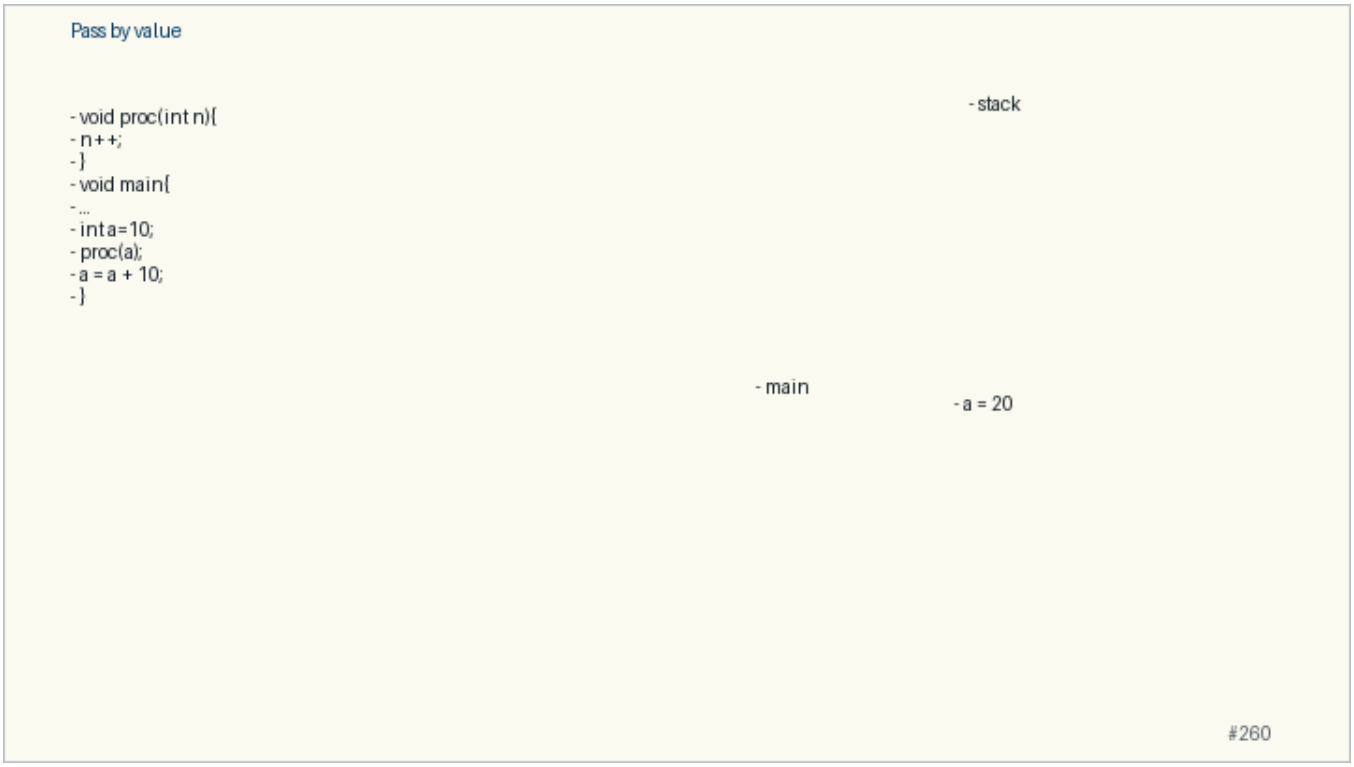
- stack
- void proc(int n){
- n++;
- }
- void main{
- ...
- int a=10;
- proc(a);
- }
- main
- a = 10



1.10 Backup: Pass by value

Original slide 260

- stack
- void proc(int n){
- n++;
- }
- void main{
- ...
- int a=10;
- proc(a);
- a = a + 10;
- }
- main
- a = 20



2 Pass by Reference

Live pacing: about 8 min

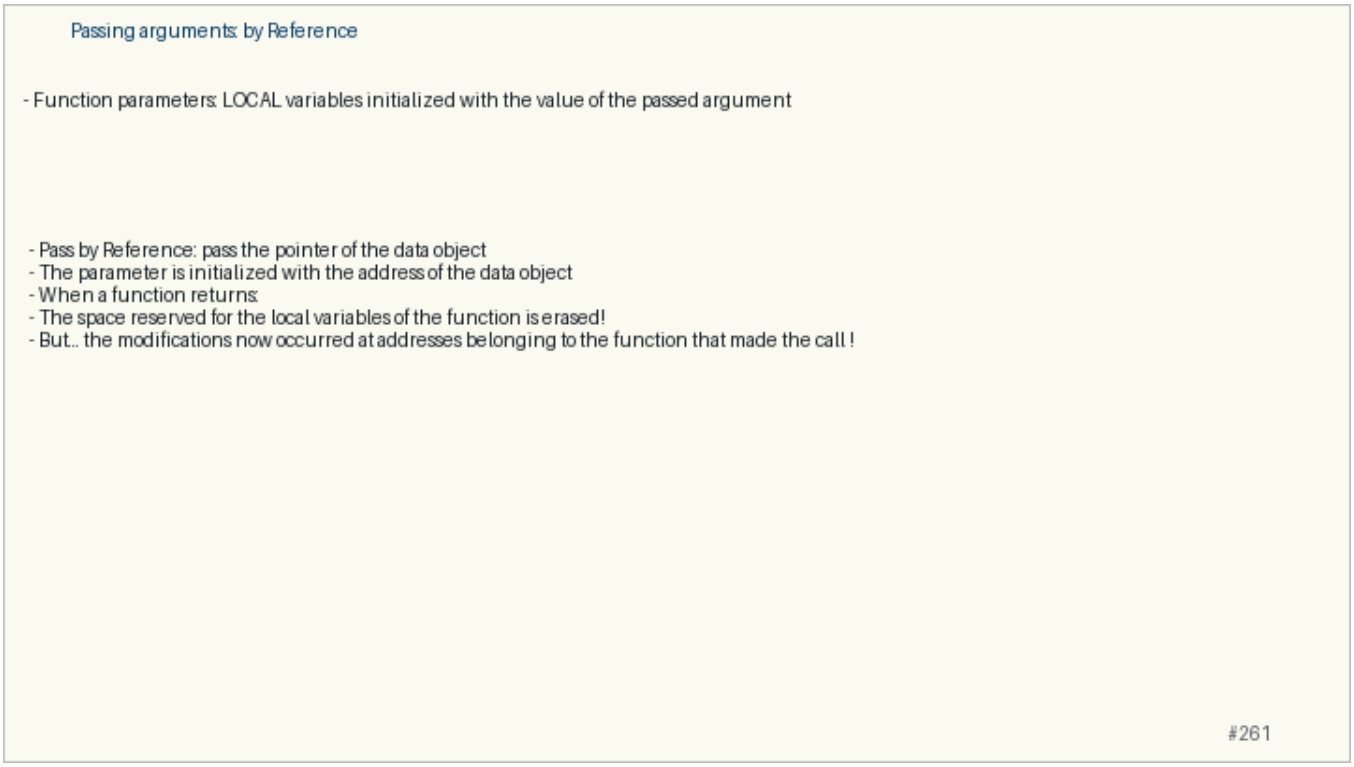
Question. What is the role of Pass by Reference in the course story?

Core claim. Pass by Reference is one step in the path from source text to reliable execution.

Target capability. Students can connect the topic to a concrete command, program state, or memory state.

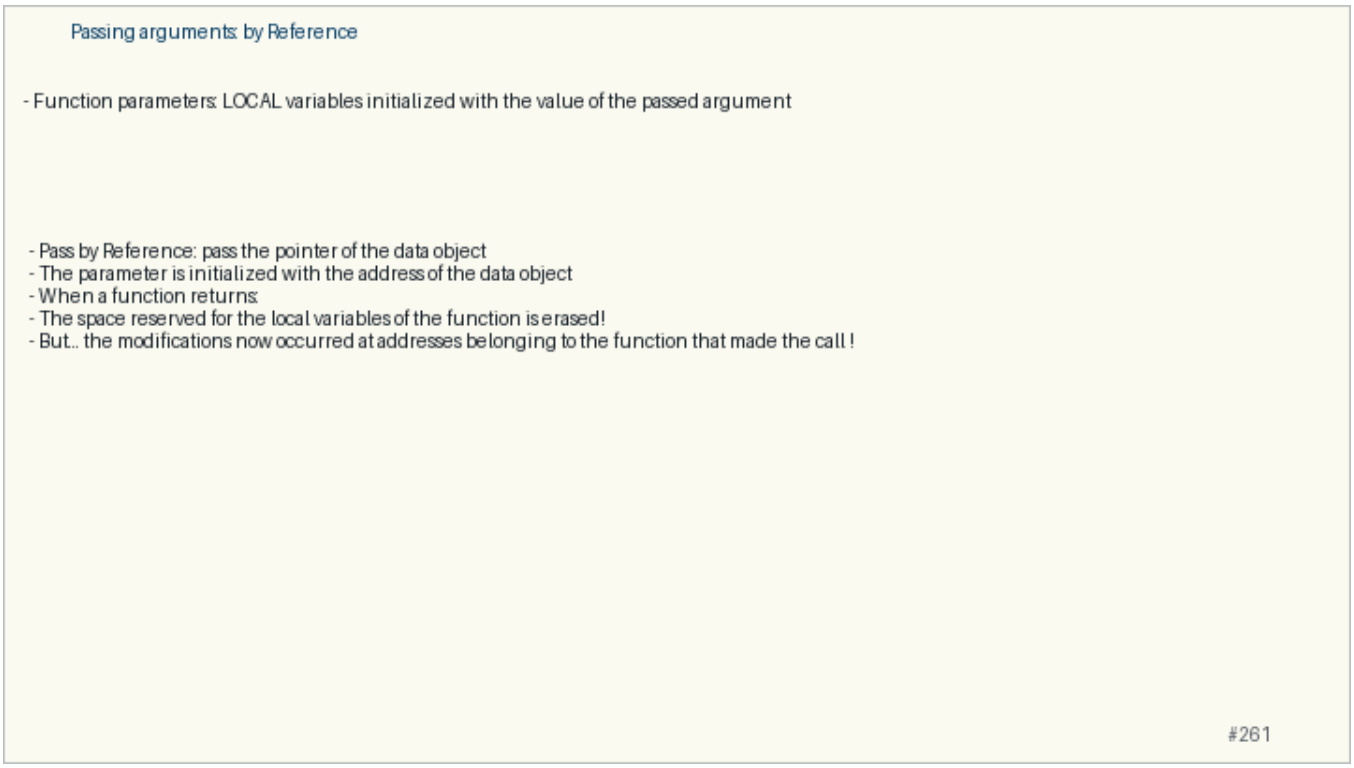
2.1 Main Path

- Start from the question: What is the role of Pass by Reference in the course story?
- Build the model: Pass by Reference is one step in the path from source text to reliable execution.
- End with a checkable action: Students can connect the topic to a concrete command, program state, or memory state.
- Keep only this slide on the horizontal path during the live lecture.



2.2 Worked Example

- Use this as the live trace: Use one short trace, then ask students which state changed and which state did not.
- Representative source slide: 261
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.



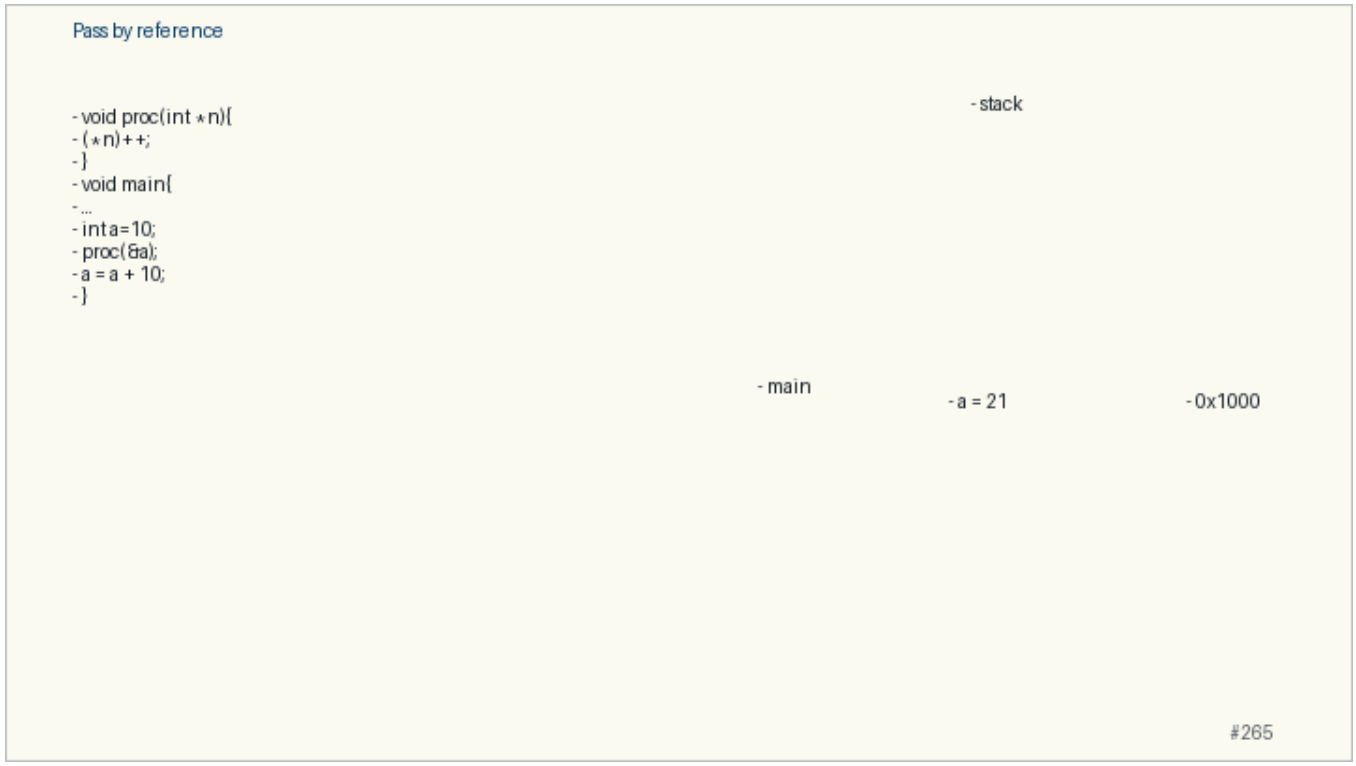
2.3 Anecdote Or Motivation

Keep the discussion anchored in observable behavior: command output, compiler message, or memory drawing.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

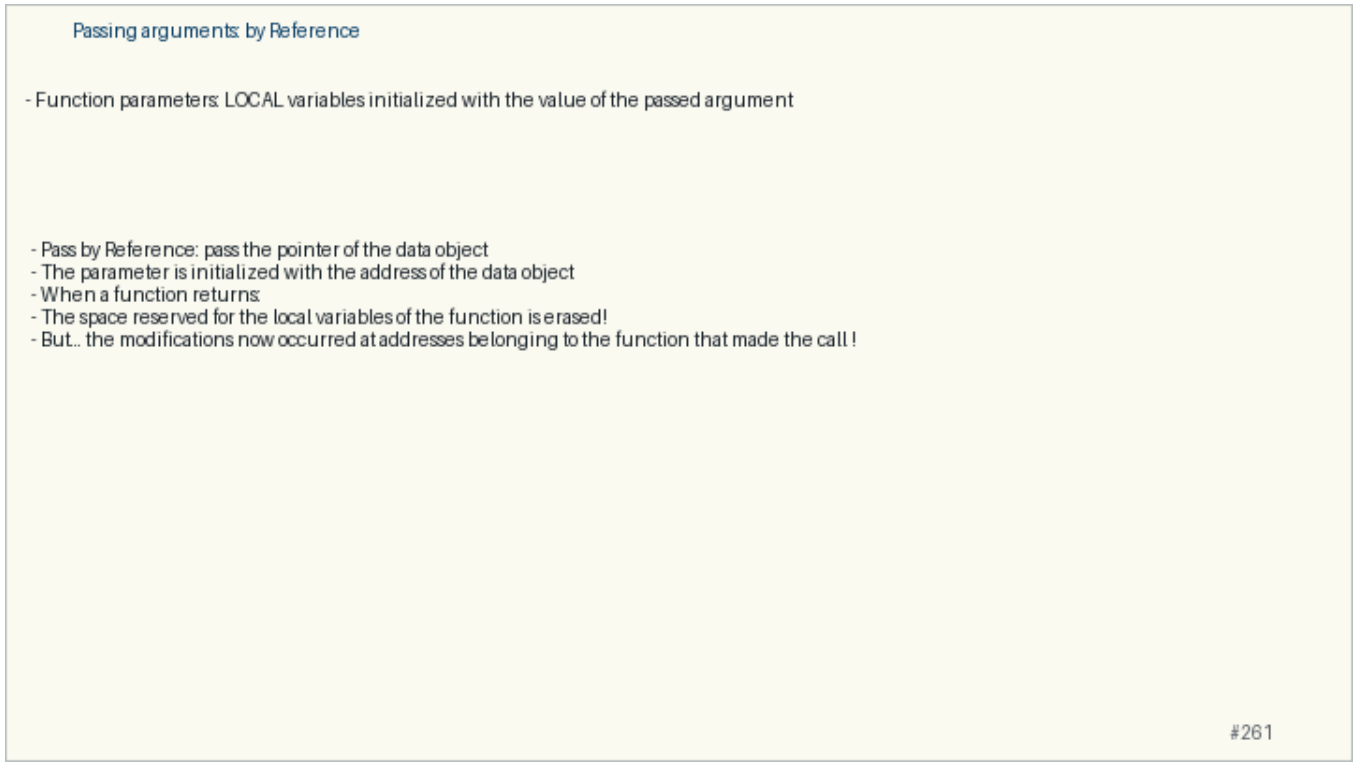
2.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.



2.5 Backup Index

Original slides 261-265



slide 261

Pass by reference

```
- void proc(int *n){
- (*n)++;
- void main{
- ...
- int a=10;
- proc(&a);
- }
```

- stack
- n = 0x1000
- proc
- main
- a = 10
- 0x1000

#262

slide 262

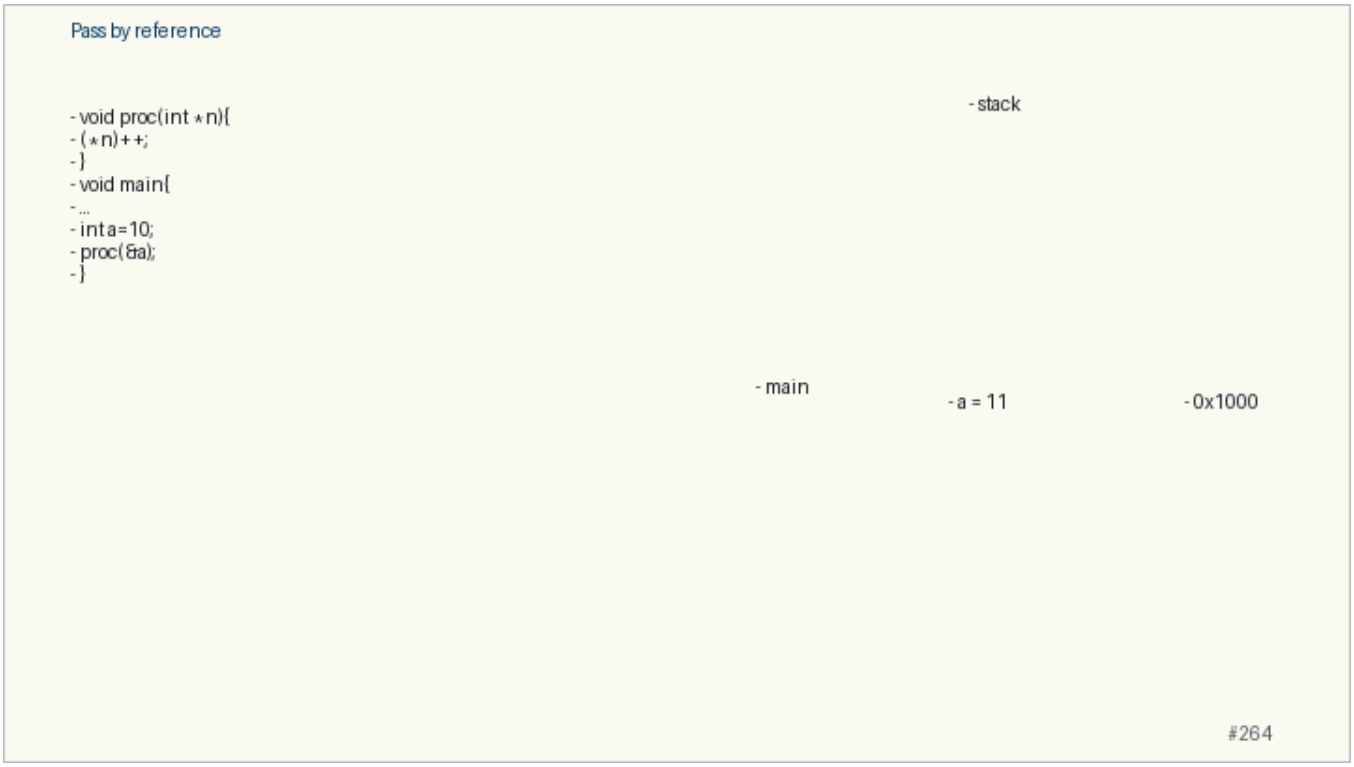
Pass by reference

```
- void proc(int *n){
- (*n)++;
- void main{
- ...
- int a=10;
- proc(&a);
- }
```

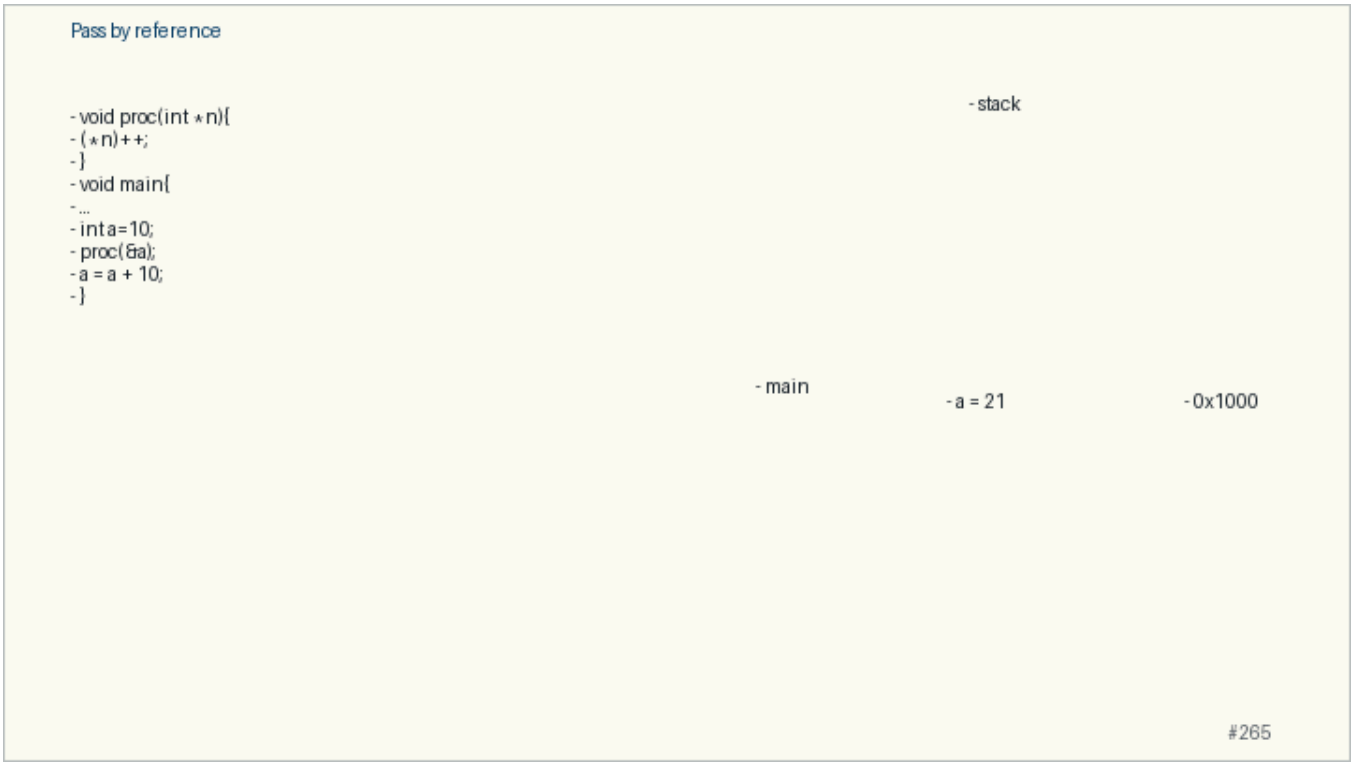
- stack
- n = 0x1000
- proc
- main
- a = 11
- 0x1000

#263

slide 263



slide 264



slide 265

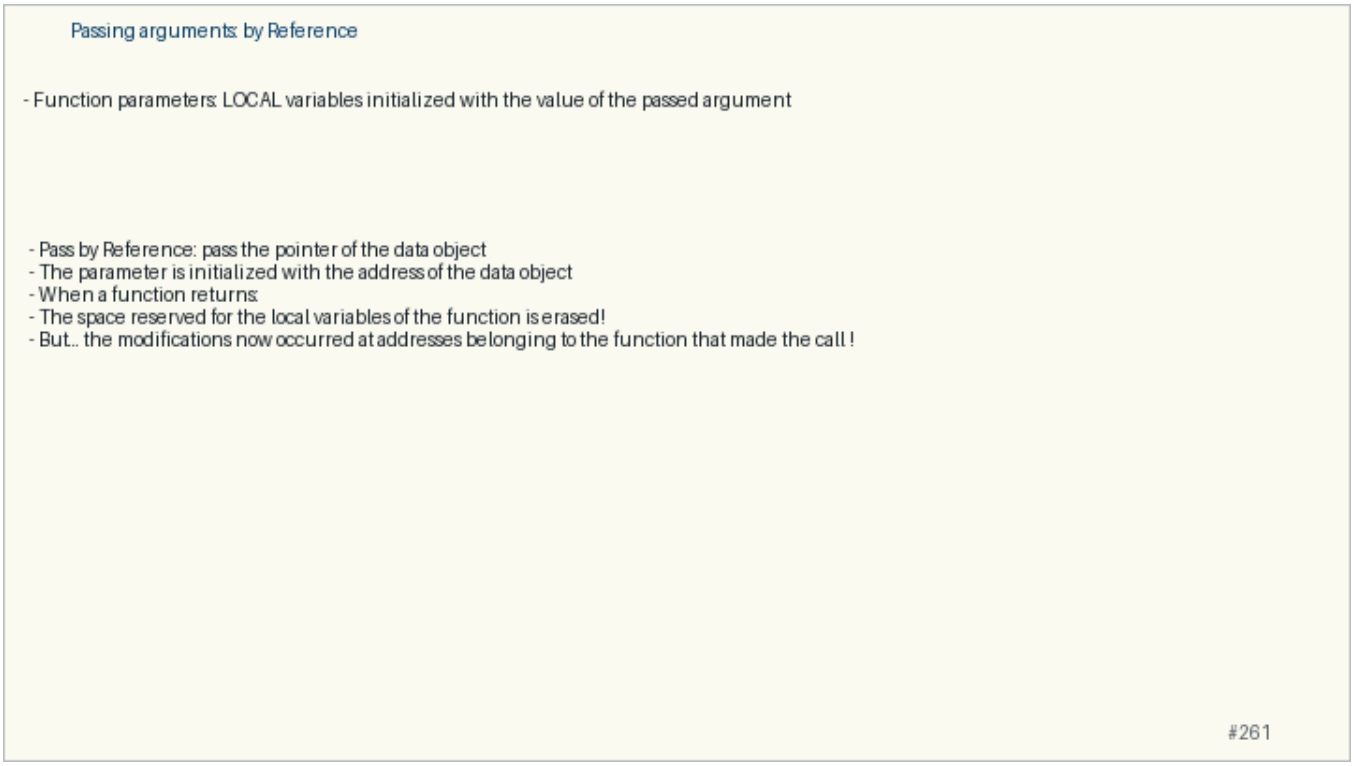
Anchors: Passing arguments: by Reference; Pass by reference

2.6 Backup: Passing arguments: by Reference

Original slide 261

- Function parameters: LOCAL variables initialized with the value of the passed argument
- Pass by Reference: pass the pointer of the data object
- The parameter is initialized with the address of the data object

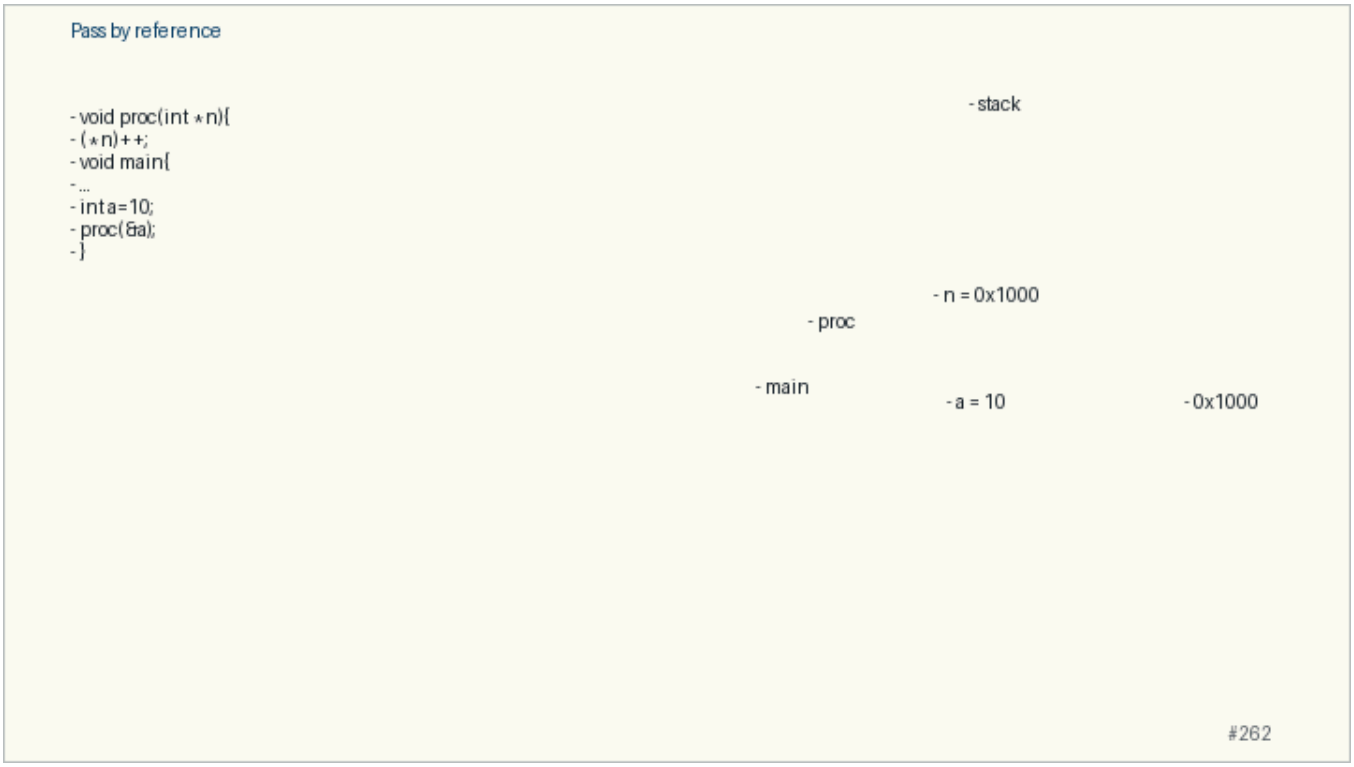
- When a function returns:
- The space reserved for the local variables of the function is erased!
- But… the modifications now occurred at addresses belonging to the function that made the call !



2.7 Backup: Pass by reference

Original slide 262

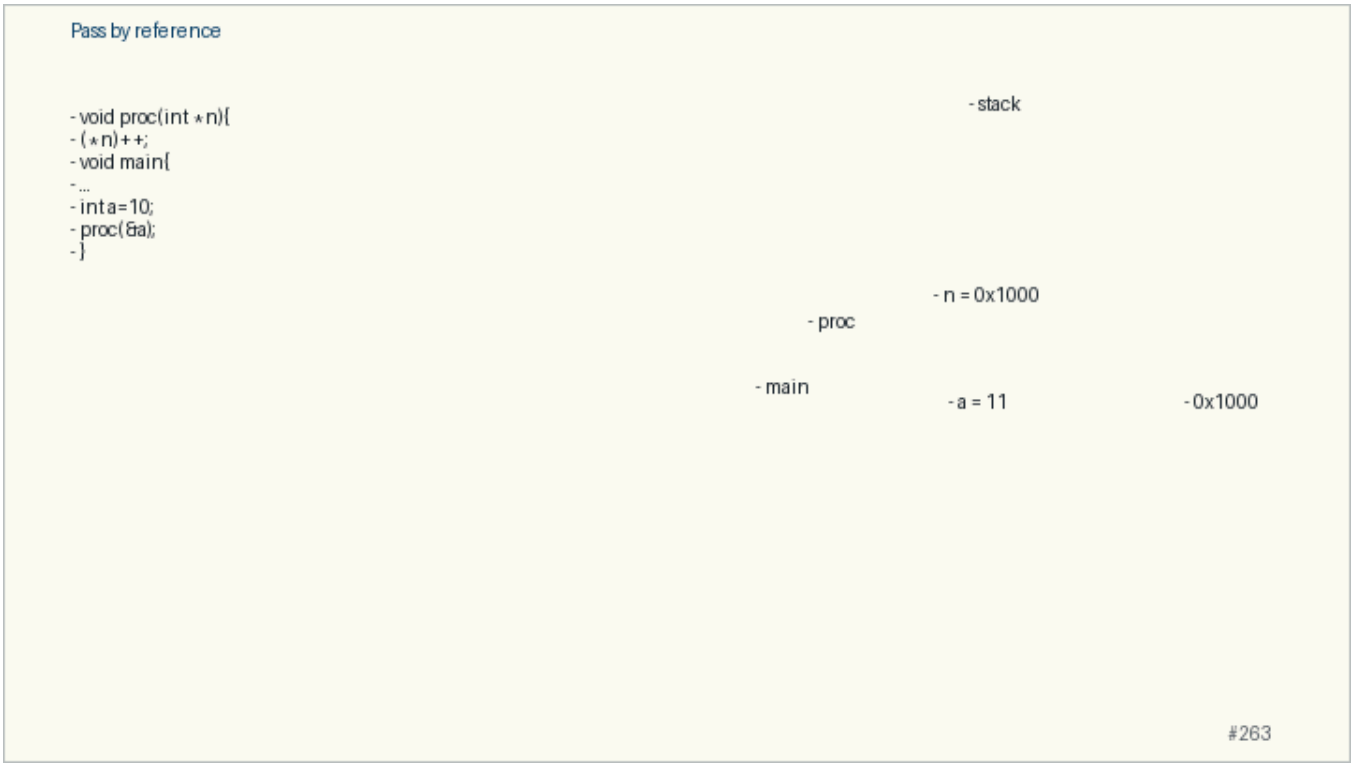
- stack
- void proc(int *n){
- (*n)++;
- void main{
- …
- int a=10;
- proc(&a);
- }
- n = 0x1000
- proc
- main
- a = 10
- 0x1000



2.8 Backup: Pass by reference

Original slide 263

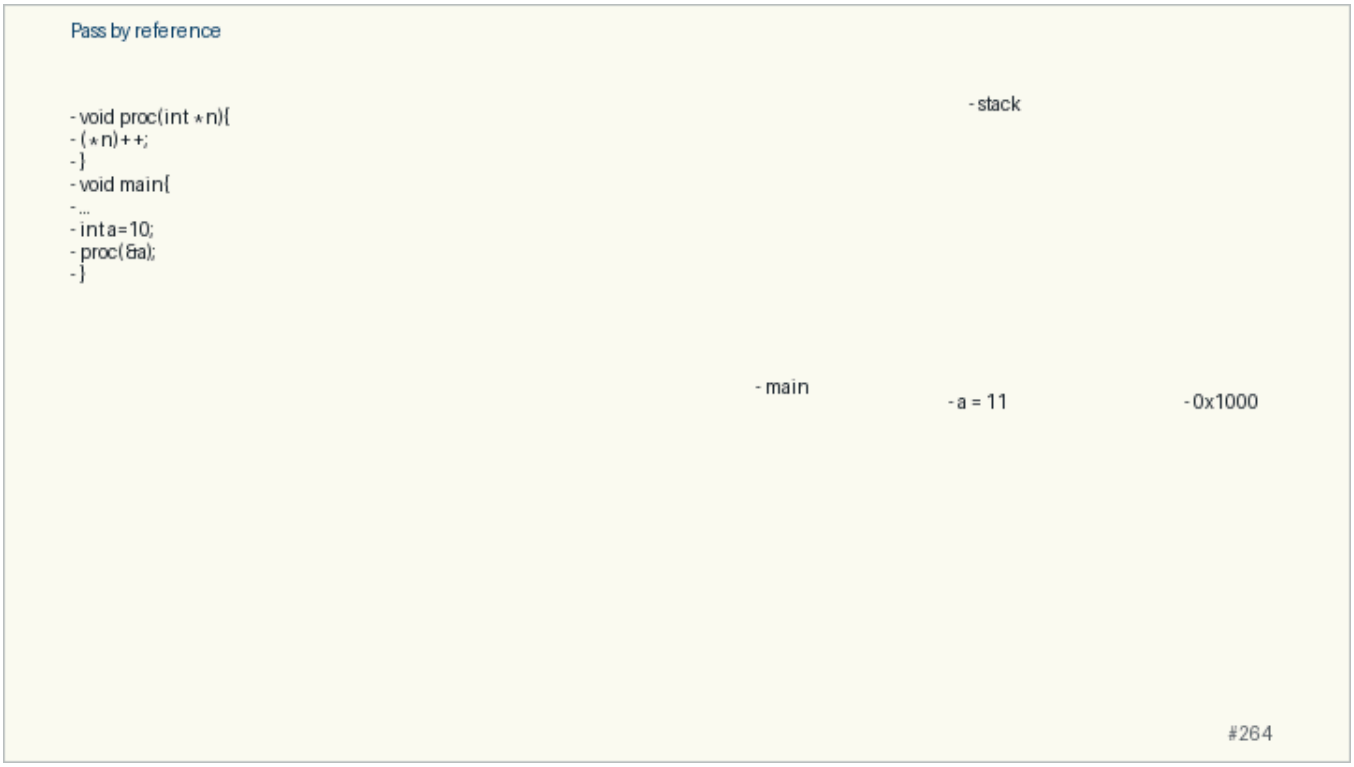
- stack
- void proc(int *n){
- (*n)++;
- void main{
- ...
- int a=10;
- proc(&a);
- }
- n = 0x1000
- proc
- main
- a = 11
- 0x1000



2.9 Backup: Pass by reference

Original slide 264

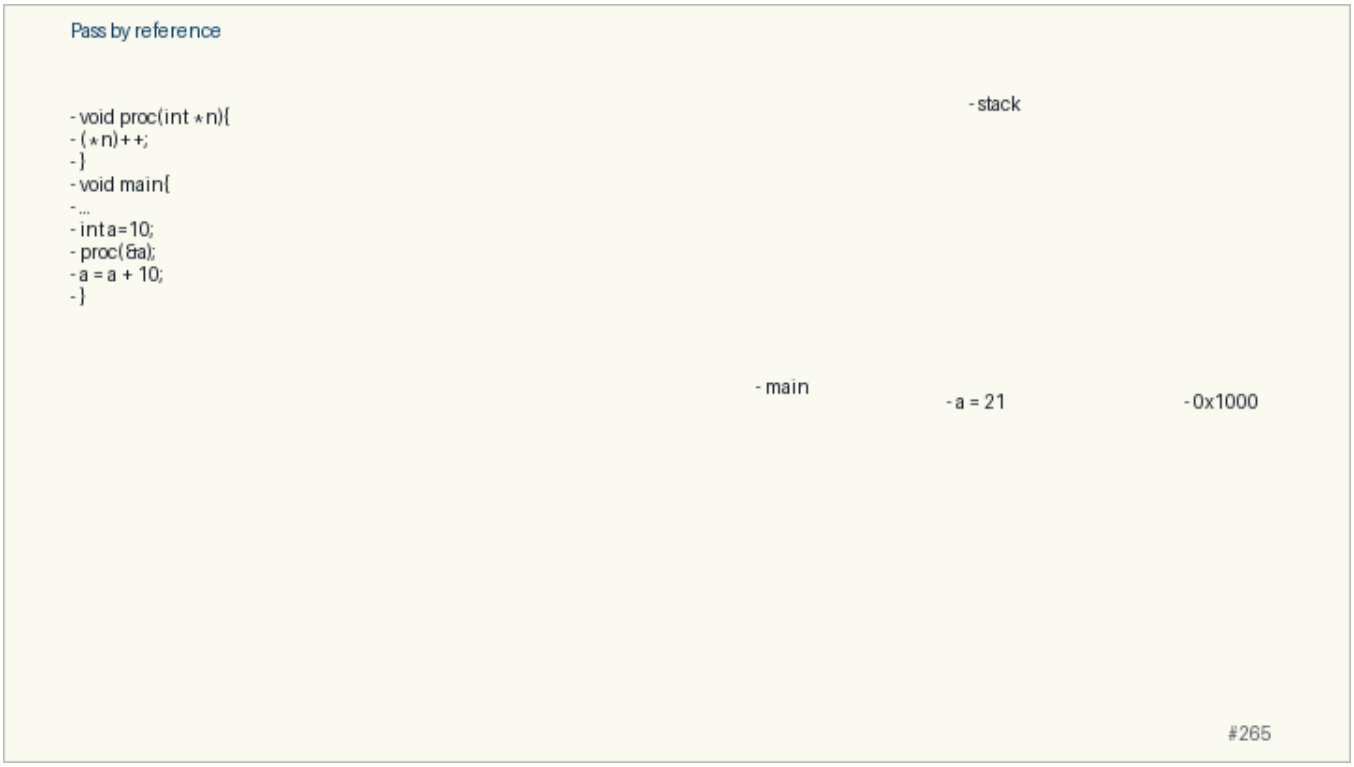
- stack
- void proc(int *n){
- (*n)++;
- }
- void main{
- ...
- int a=10;
- proc(&a);
- }
- main
- a = 11
- 0x1000



2.10 Backup: Pass by reference

Original slide 265

- stack
- void proc(int *n){
- (*n)++;
- }
- void main{
- ...
- int a=10;
- proc(&a);
- a = a + 10;
- }
- main
- a = 21
- 0x1000



3 Pointer Parameters

Live pacing: about 8 min

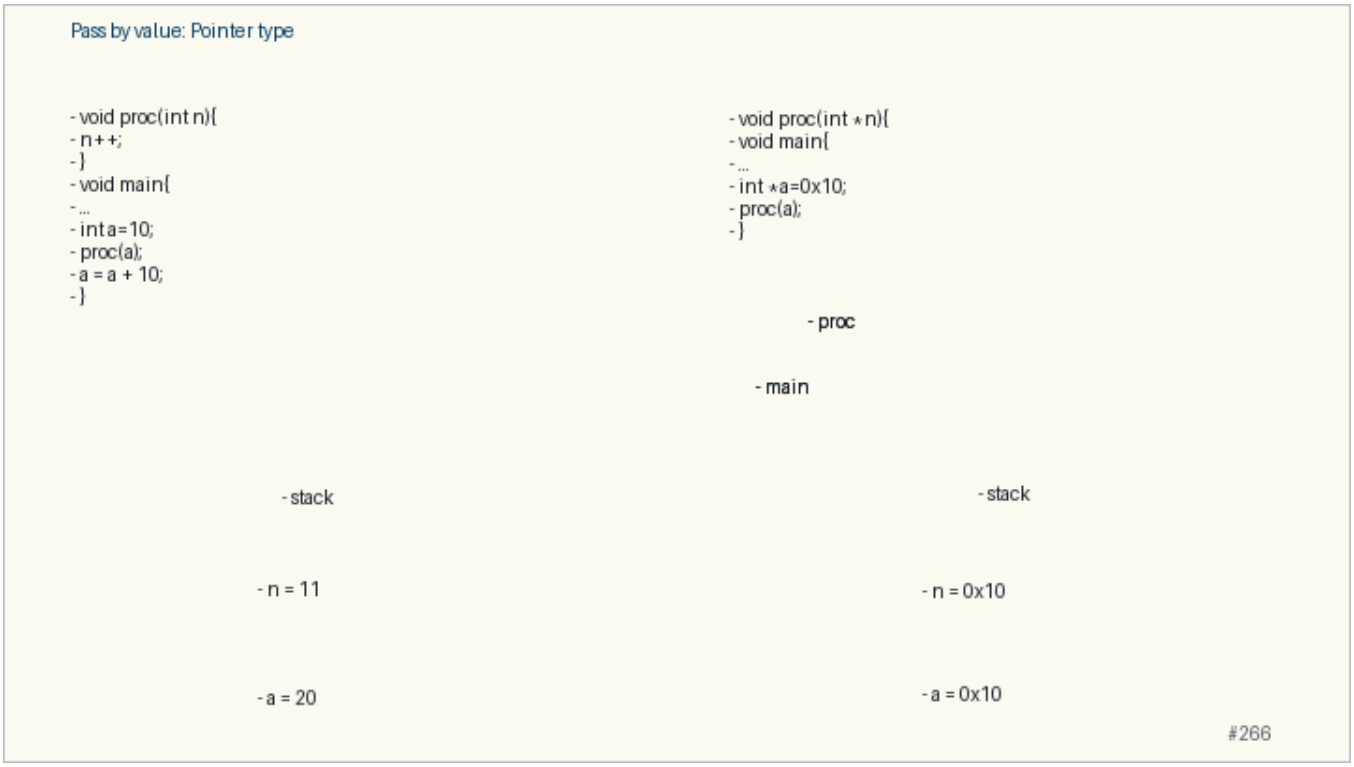
Question. What does a pointer value name?

Core claim. A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.

Target capability. Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.

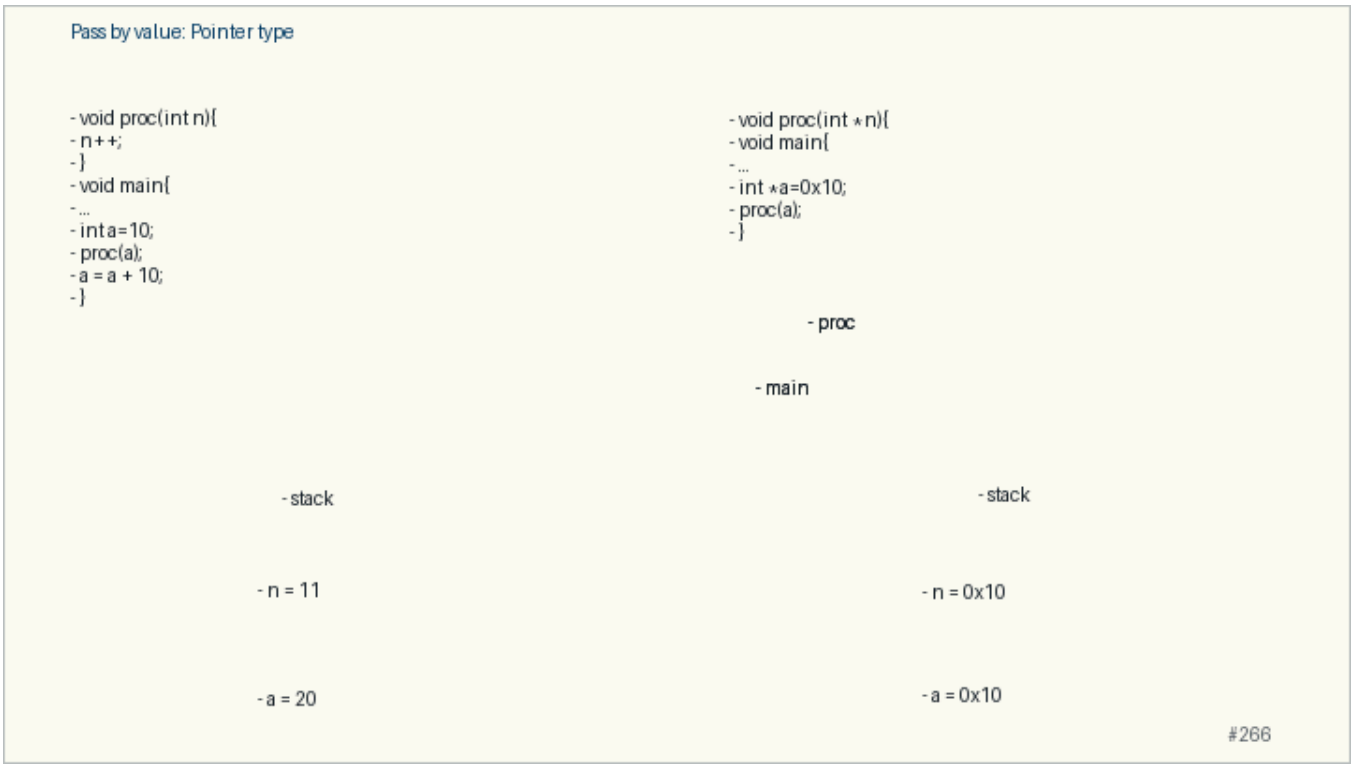
3.1 Main Path

- Start from the question: What does a pointer value name?
- Build the model: A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.
- End with a checkable action: Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.
- Keep only this slide on the horizontal path during the live lecture.



3.2 Worked Example

- Use this as the live trace: Trace `int x`, `int *p = &x`, then mutate `x` through both names.
- Representative source slide: 266
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.



3.3 Anecdote Or Motivation

Pointers feel hard because one line can talk about two objects: the pointer variable and the pointee.

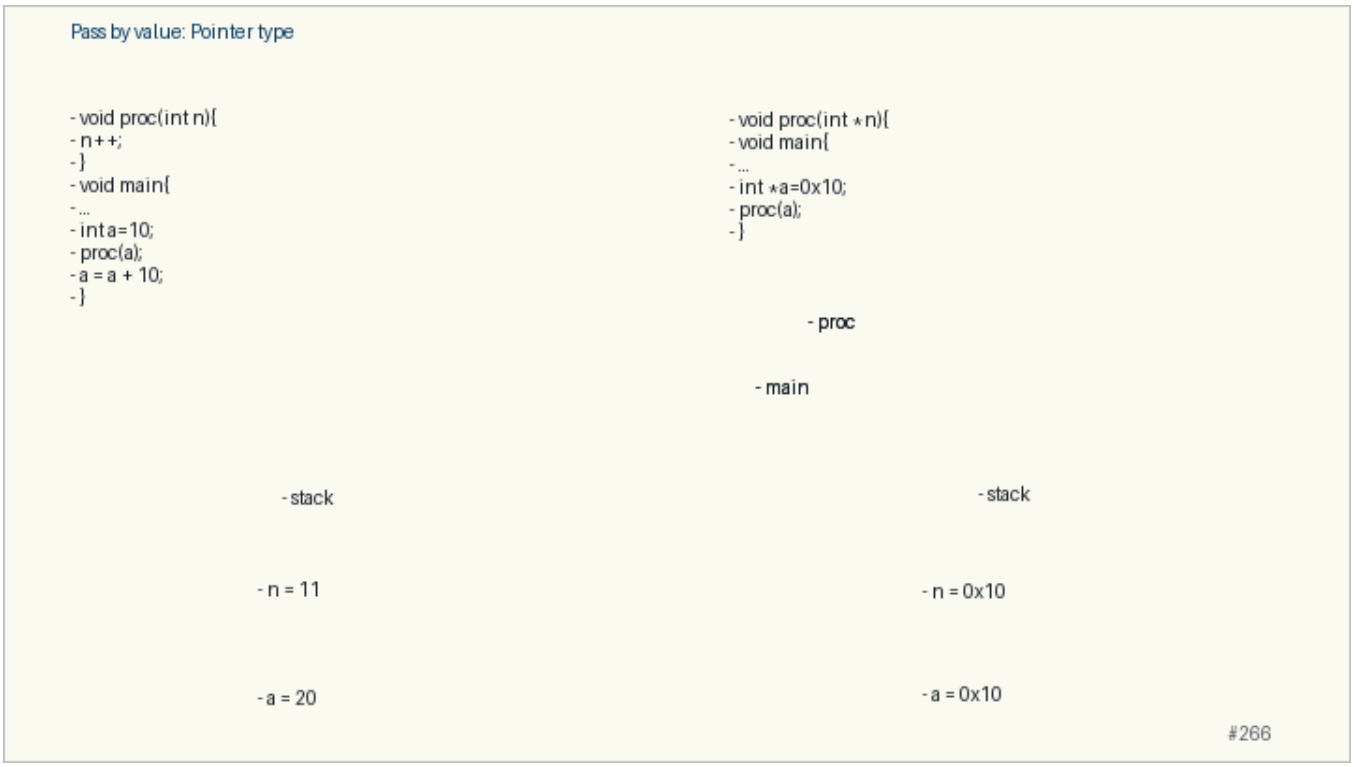
- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

3.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

3.5 Backup Index

Original slides 266-274



slide 266

Pass by value: Pointer type

```
- void proc(int n){
- n++;
- }
- void main{
- ...
- int a=10;
- proc(a);
- a = a + 10;
- }
```

```
- void proc(int *n){
- n++;
- void main{
- ...
- int *a=0x10;
- proc(a);
- }
```

- proc

- main

- stack

- n = 11

- a = 20

- stack

- n = 0x14

- a = 0x10

#267

slide 267

Pass by value: Pointer type

```
- void proc(int n){
- n++;
- }
- void main{
- ...
- int a=10;
- proc(a);
- a = a + 10;
- }
```

```
- void proc(int *n){
- n++;
- }
- void main{
- ...
- int *a=0x10;
- proc(a);
- }
```

- proc

- main

- stack

- n = 11

- a = 20

- stack

- a = 0x10

#268

slide 268

Pass by value: Pointer type

```
- void proc(int n){
- n++;
- }
- void main{
- ...
- int a=10;
- proc(a);
- a = a + 10;
- }
```

```
- void proc(int *n){
- n++;
- }
- void main{
- ...
- int *a=0x10;
- proc(a);
- a = a + 0x10;
- }
```

- proc

- main

- stack

- n = 11

- a = 20

- stack

- a = 0x20

#269

slide 269

Pass by reference: Pointer

```
- void proc(int *n){
- (*n)++;
- }
- void main{
- ...
- int a=10;
- proc(&a);
- a = a + 10;
- }
```

```
- void proc(int **n){
- (*n)++;
- void main{
- ...
- int *a=0x10;
- proc(&a);
- }
```

- proc

- main

- stack

- n = 0x1000

- a = 21

- stack

- n = 0x1000

- a = 0x10

- 0x1000

#270

slide 270

slide 271

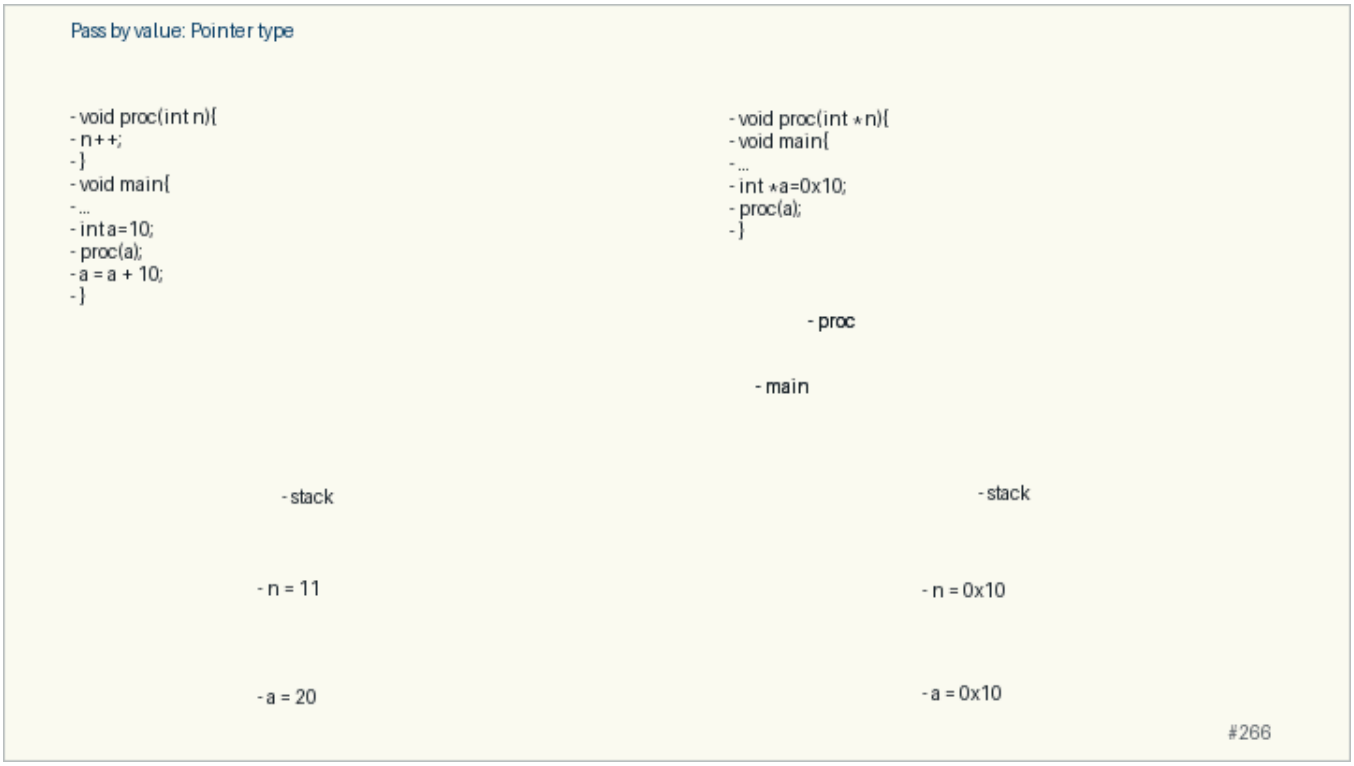
Anchors: Pass by value: Pointer type; Pass by reference: Pointer; Let's…

3.6 Backup: Pass by value: Pointer type

Original slide 266

- void proc(int n){
- n++;
- }
- void main{
- …
- int a=10;
- proc(a);
- a = a + 10;
- }
- void proc(int *n){
- void main{
- …
- int *a=0x10;
- proc(a);
- }
- proc
- proc
- main
- main

- stack
- stack
- n = 11
- n = 0x10
- a = 0x10
- a = 20

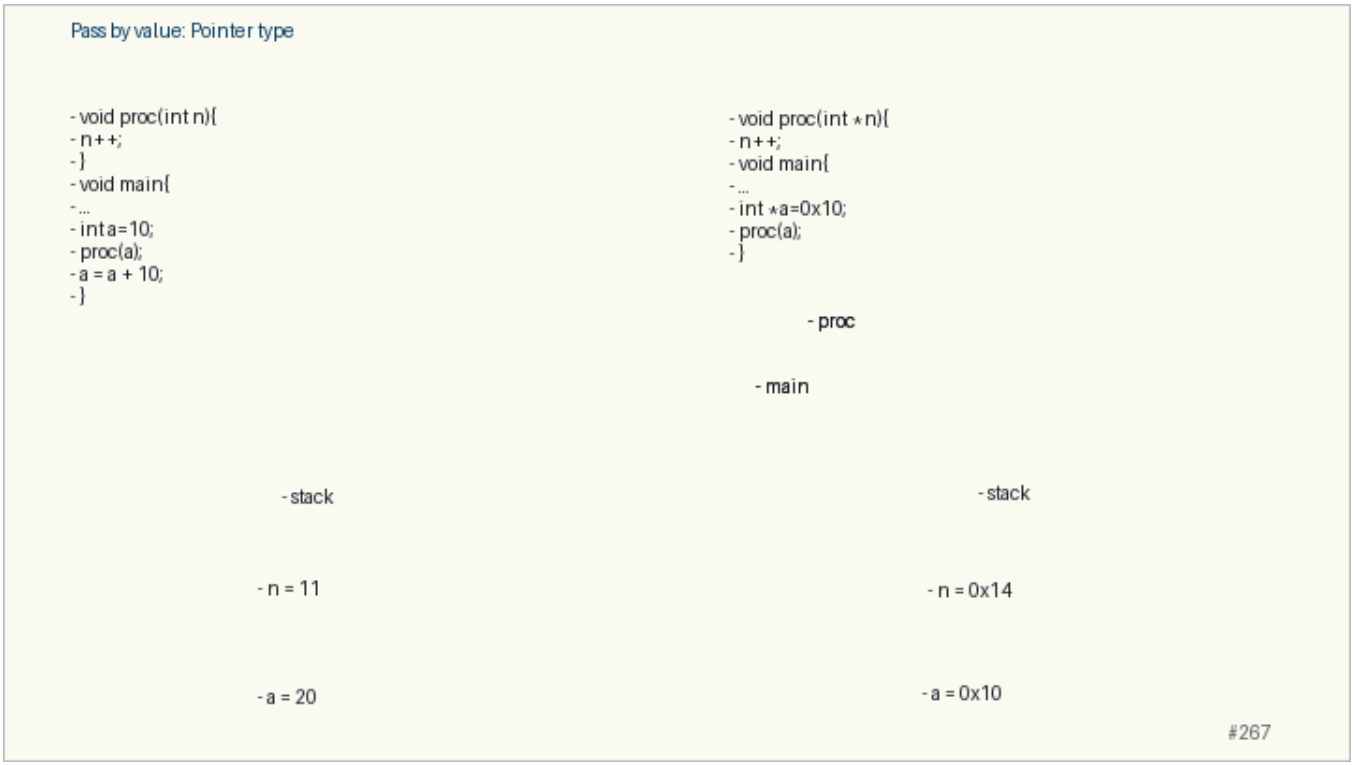


3.7 Backup: Pass by value: Pointer type

Original slide 267

- void proc(int n){
- n++;
- }
- void main{
- ...
- int a=10;
- proc(a);
- a = a + 10;
- }
- void proc(int *n){
- n++;
- void main{
- ...
- int *a=0x10;

- proc(a);
- }
- proc
- proc
- main
- main
- stack
- stack
- n = 11
- n = 0x14
- a = 0x10
- a = 20



3.8 Backup: Pass by value: Pointer type

Original slide 268

- void proc(int n){
- n++;
- }
- void main{
- ...
- int a=10;
- proc(a);
- a = a + 10;

- }
- void proc(int *n){
- n++;
- }
- void main{
- ...
- int *a=0x10;
- proc(a);
- }
- proc
- main
- main
- stack
- stack
- n = 11
- a = 0x10
- a = 20

3.9 Backup: Pass by value: Pointer type

Original slide 269

- void proc(int n){
- n++;
- }

- void main{
- …
- int a=10;
- proc(a);
- a = a + 10;
- }
- void proc(int *n){
- n++;
- }
- void main{
- …
- int *a=0x10;
- proc(a);
- a = a + 0x10;
- }
- proc
- main
- main
- stack
- stack
- n = 11
- a = 0x20
- a = 20

3.10 Backup: Pass by reference: Pointer

Original slide 270

- void proc(int **n){
- (*n)++;
- void main{
- ...
- int *a=0x10;
- proc(&a);
- }
- void proc(int *n){
- (*n)++;
- }
- void main{
- ...
- int a=10;
- proc(&a);
- a = a + 10;
- }
- proc
- proc
- main
- main

- stack
- stack
- n = 0x1000
- n = 0x1000
- a = 0x10
- 0x1000
- a = 21

3.11 Backup: Pass by reference: Pointer

Original slide 271

- void proc(int **n){
- (*n)++;
- void main{
- ...
- int *a=0x10;
- proc(&a);
- }
- void proc(int *n){
- (*n)++;
- }
- void main{
- ...
- int a=10;

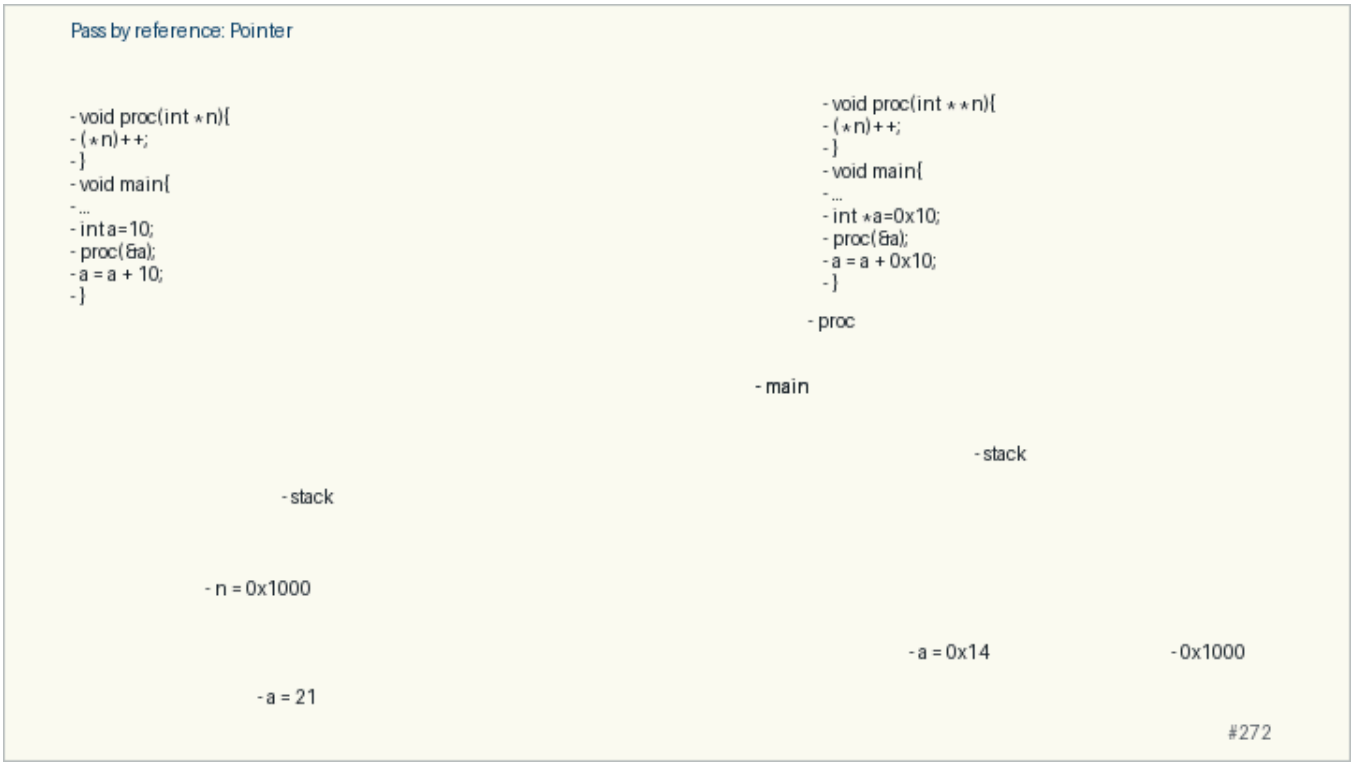
- proc(&a);
- a = a + 10;
- }
- proc
- proc
- main
- main
- stack
- stack
- n = 0x1000
- n = 0x1000
- a = 0x14
- 0x1000
- a = 21

3.12 Backup: Pass by reference: Pointer

Original slide 272

- void proc(int **n){
- (*n)++;
- }
- void main{
- ...
- int *a=0x10;

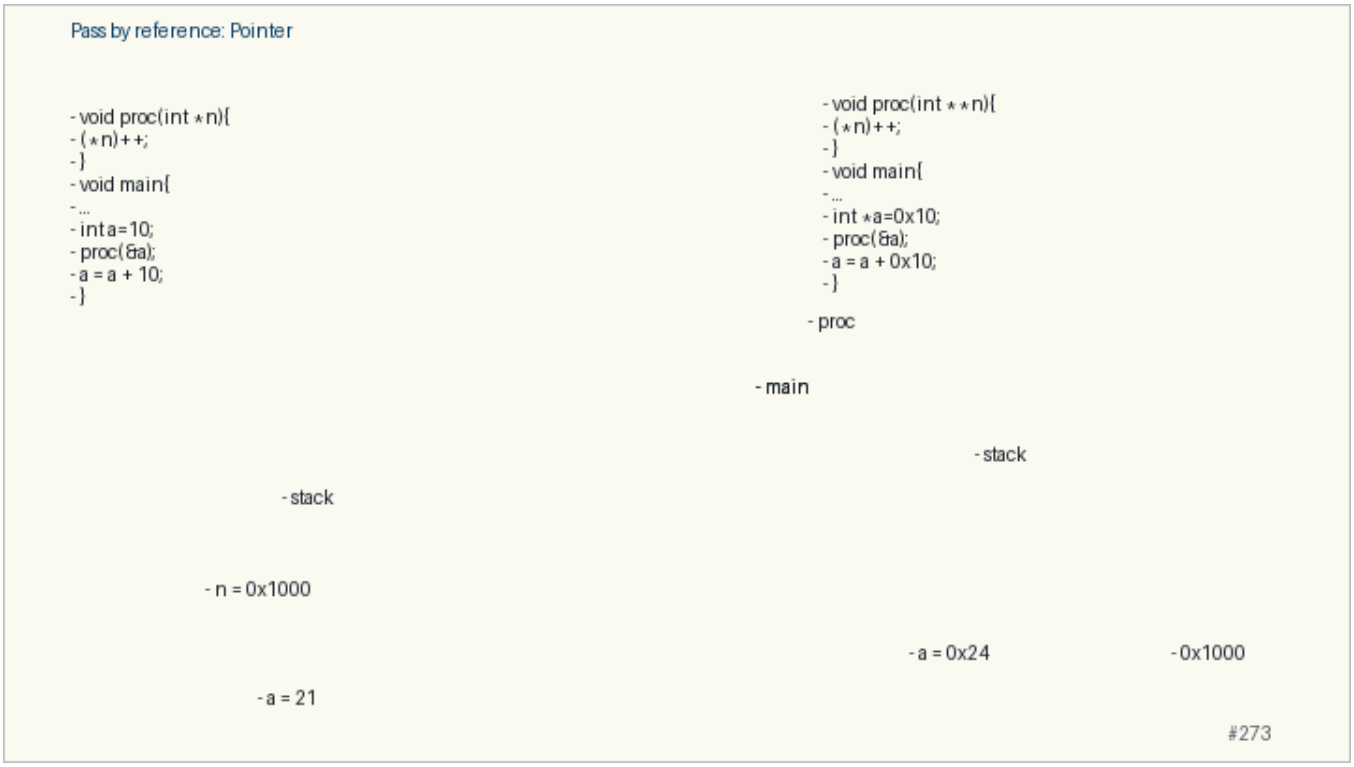
- proc(&a);
- a = a + 0x10;
- }
- void proc(int *n){
- (*n)++;
- }
- void main{
- ...
- int a=10;
- proc(&a);
- a = a + 10;
- }
- proc
- main
- main
- stack
- stack
- n = 0x1000
- a = 0x14
- 0x1000
- a = 21



3.13 Backup: Pass by reference: Pointer

Original slide 273

- void proc(int **n){
- (*n)++;
- }
- void main{
- …
- int *a=0x10;
- proc(&a);
- a = a + 0x10;
- }
- void proc(int *n){
- (*n)++;
- }
- void main{
- …
- int a=10;
- proc(&a);
- a = a + 10;
- }
- proc
- main
- main
- stack
- stack
- n = 0x1000
- a = 0x24
- 0x1000
- a = 21



3.14 Backup: Let's...

Original slide 274

- Kahoot time!
- Q5- Q9

4 Array Parameters

Live pacing: about 8 min

Question. When does an array behave like a pointer, and when does it not?

Core claim. Arrays are contiguous objects; many expressions expose the address of their first element.

Target capability. Students can draw array storage, pointer arithmetic, strings, and 2D indexing.

4.1 Main Path

- Start from the question: When does an array behave like a pointer, and when does it not?
- Build the model: Arrays are contiguous objects; many expressions expose the address of their first element.
- End with a checkable action: Students can draw array storage, pointer arithmetic, strings, and 2D indexing.
- Keep only this slide on the horizontal path during the live lecture.

4.2 Worked Example

- Use this as the live trace: Use `arr`, `&arr[0]`, `arr + 1`, then repeat with a string ending in `\0`.
- Representative source slide: 276
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

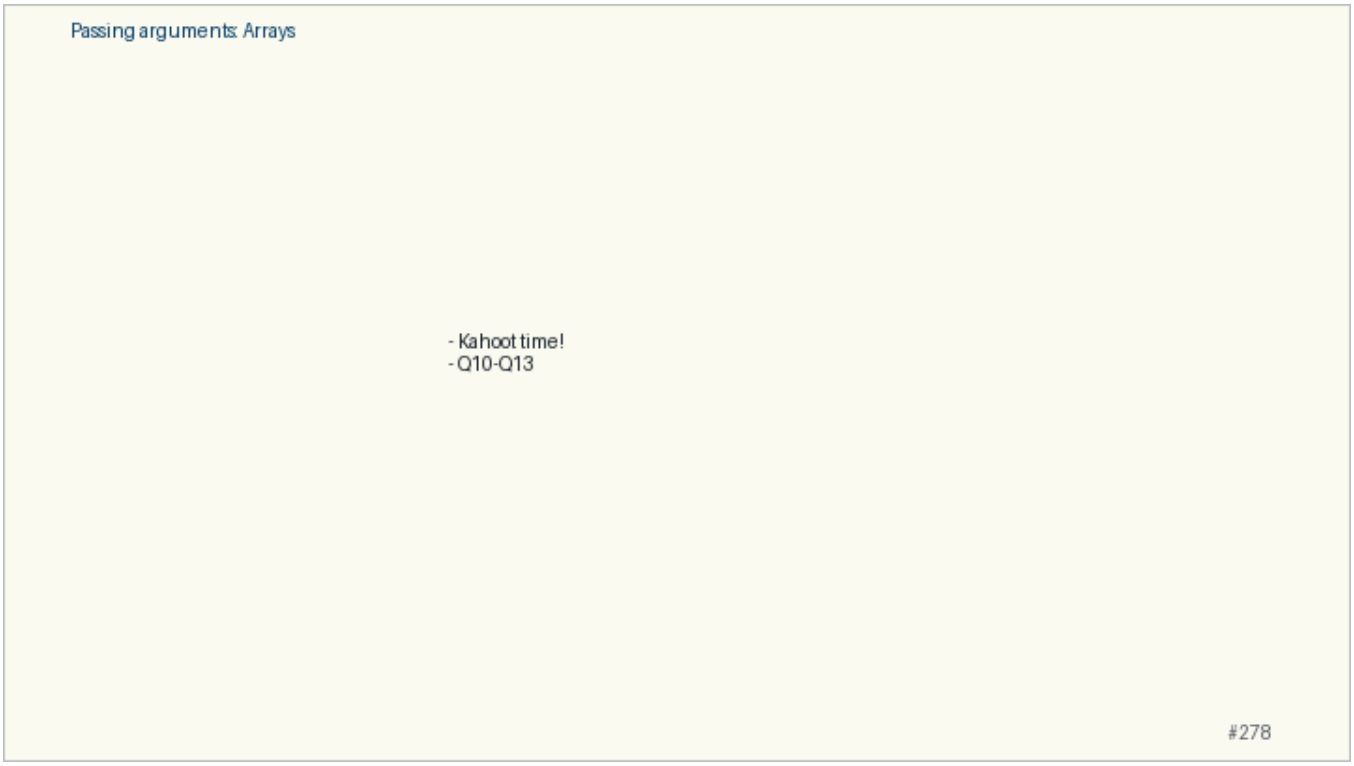
4.3 Anecdote Or Motivation

A string is not magic in C; it is an array convention plus library functions that respect it.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

4.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

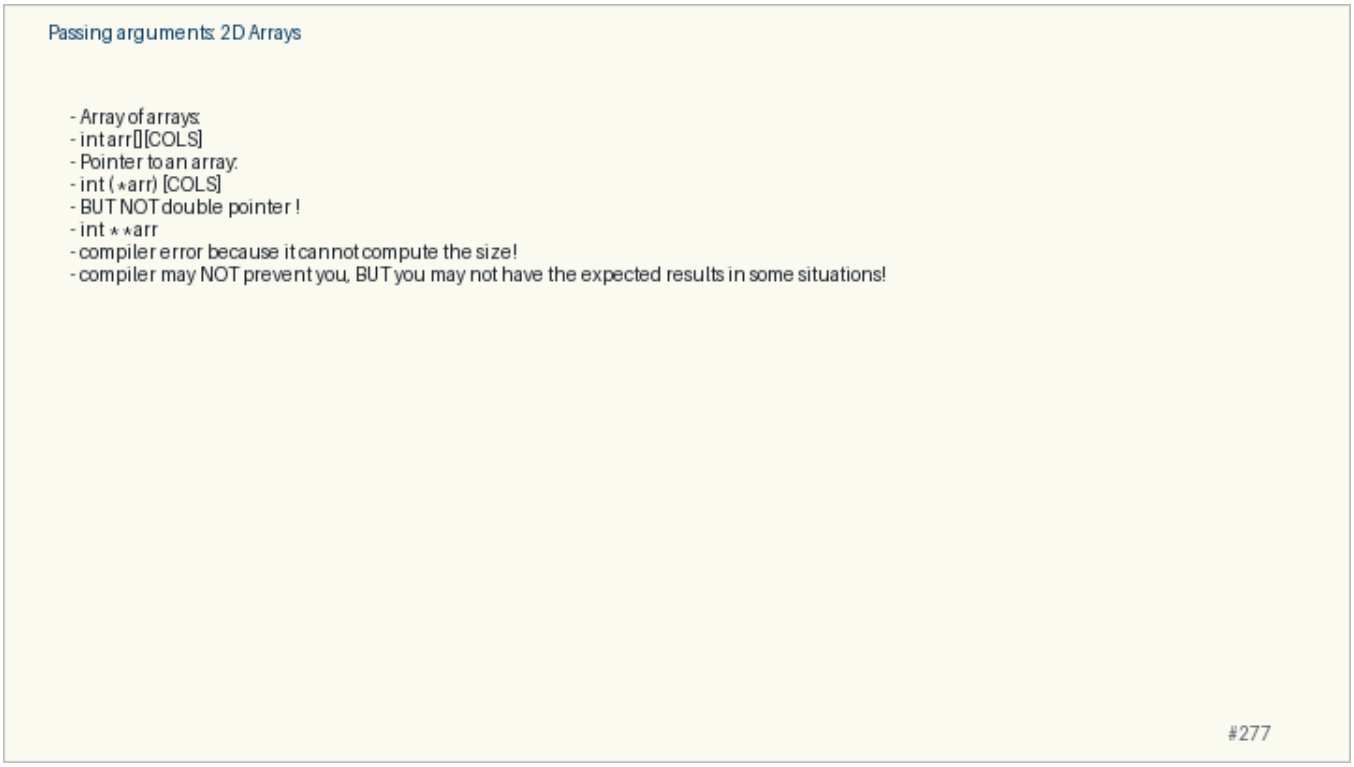


4.5 Backup Index

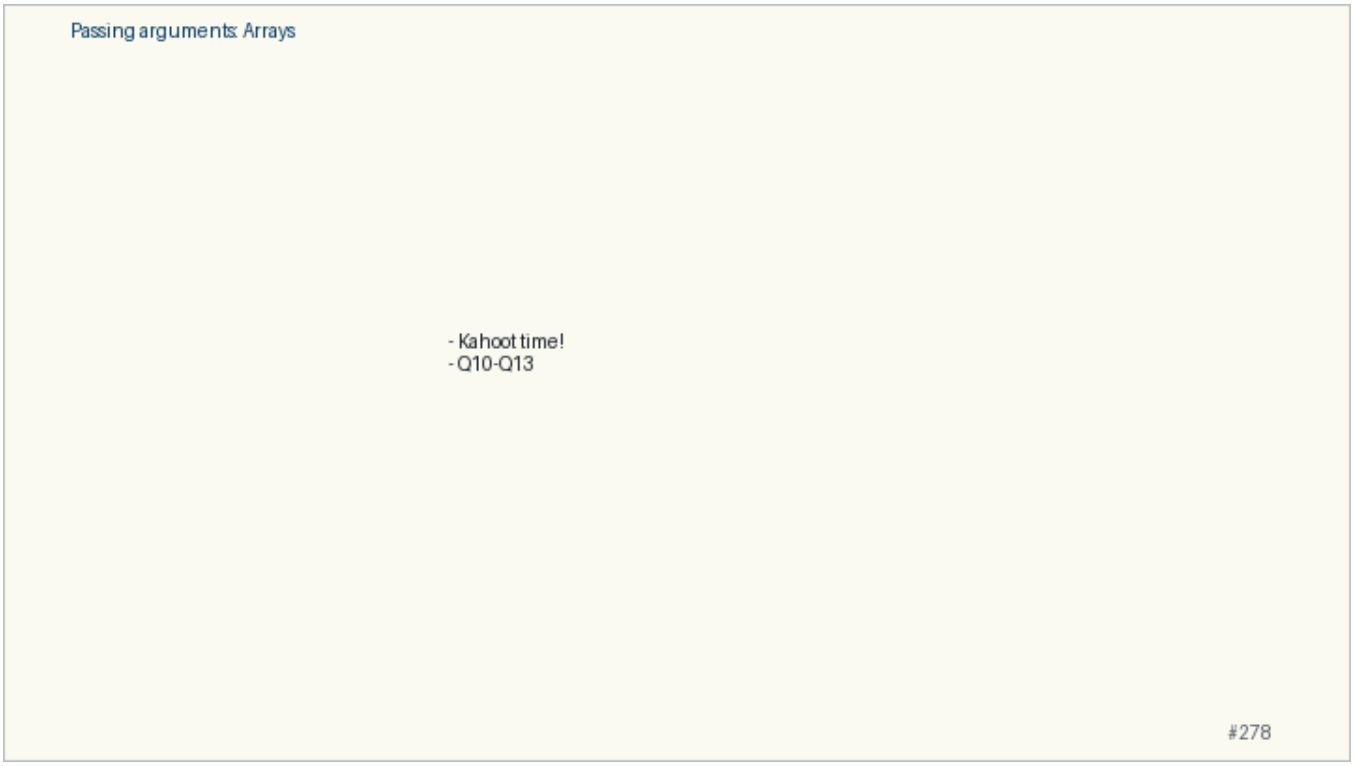
Original slides 275-278

slide 275

slide 276



slide 277



slide 278

Anchors: Passing arguments: Array; Passing arguments: 2D Arrays; Passing arguments: Arrays

4.6 Backup: Passing arguments: Array

Original slide 275

- When an array is a function parameter, the compiler translates it as a pointer
- void foo(int arr[]) { ... } -> void foo(int *arr) { ... }
- So, inside the function we cannot know the size of the array! All we have is a pointer ...

- If you try to compute the array's size using sizeof(arr), you just get the size of a pointer,
- e.g. 4 in a code compiled for a machine with 32 bits as addresses
- It's important to pass the array size as a parameter.
- void foo(int arr[], int size) { ... }

4.7 Backup: Passing arguments: Array

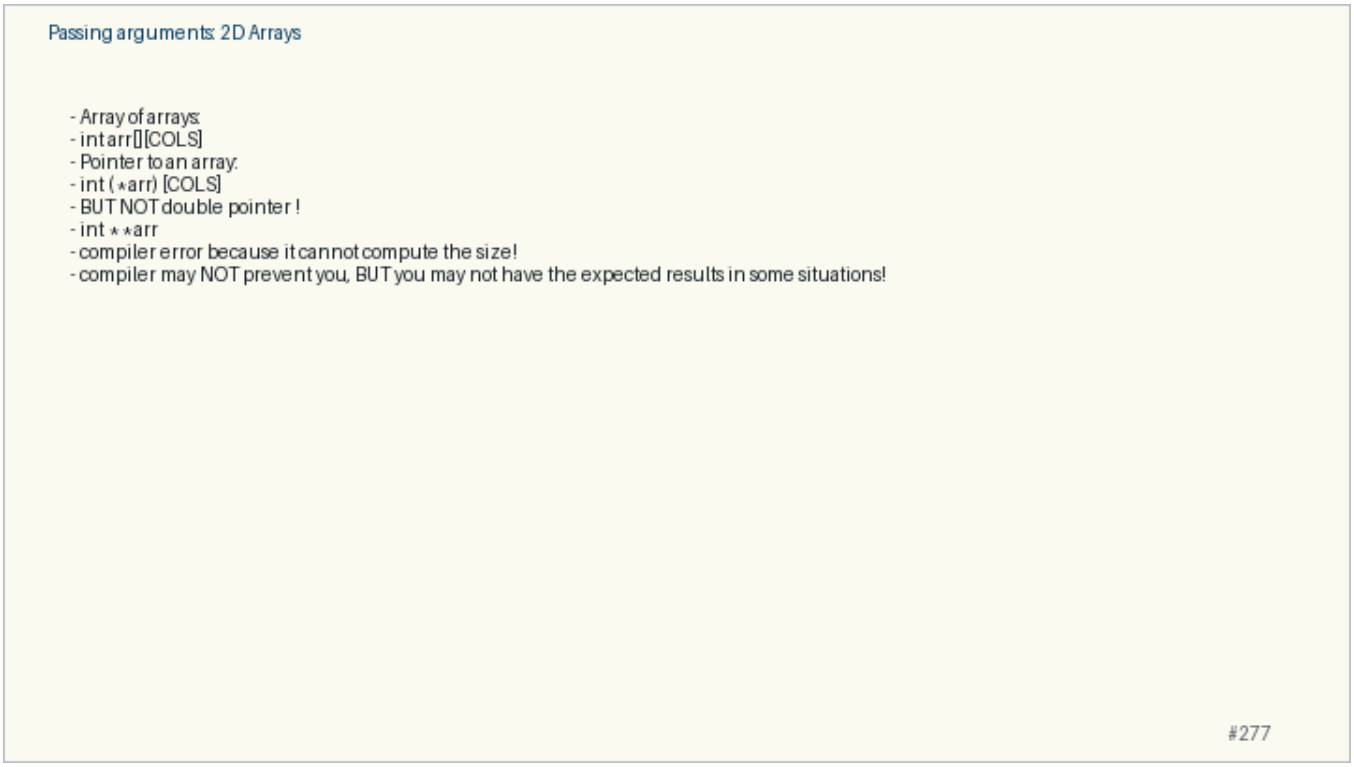
Original slide 276

```
The parameter equals to the address of the 1er element (arr=&arr[0])
Since the parameter is the address of the array, we can modify the array elements
The result is visible outside of the function
Example
Function to initialize the n values of an 1D array arr void init_vector (int arr [], int n)
{ int i; for (i=0;i<n;i++) { arr[i]=0; } }
```

4.8 Backup: Passing arguments: 2D Arrays

Original slide 277

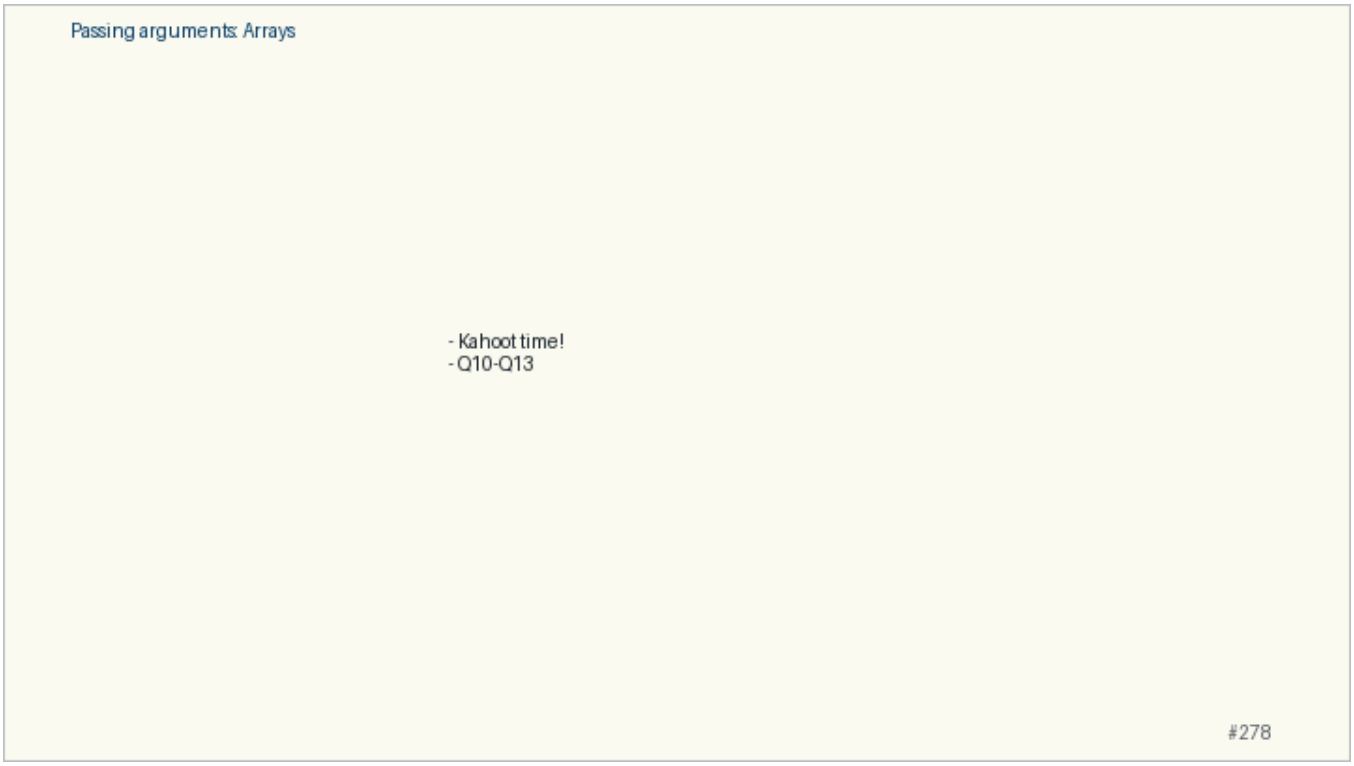
- Array of arrays:
- int arr[][COLS]
- Pointer to an array:
- int (*arr) [COLS]
- BUT NOT double pointer !
- int **arr
- compiler error because it cannot compute the size!
- compiler may NOT prevent you, BUT you may not have the expected results in some situations!



4.9 Backup: Passing arguments: Arrays

Original slide 278

- Kahoot time!
- Q10-Q13



5 Main Arguments

Live pacing: about 8 min

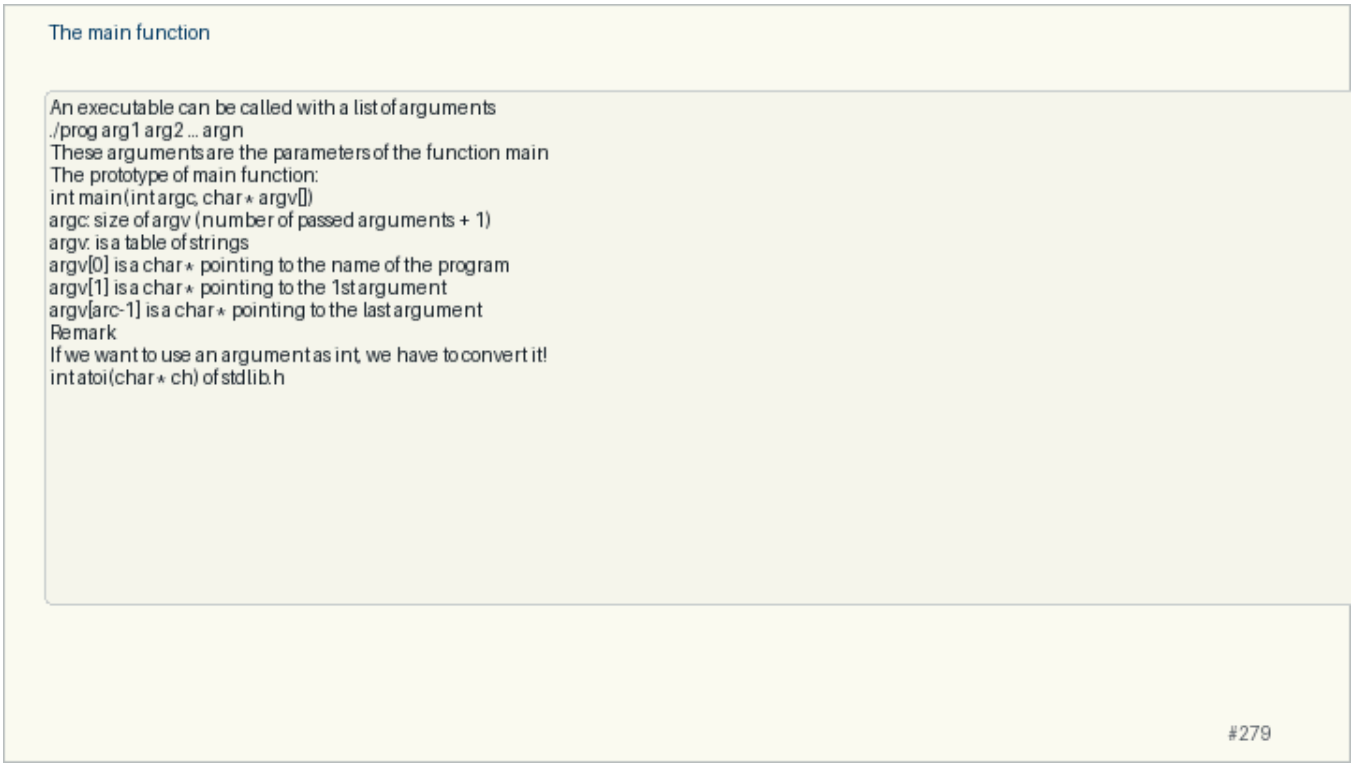
Question. What moves during a function call?

Core claim. C passes values; changing a caller object requires passing an address to that object.

Target capability. Students can explain stack frames, prototypes, array parameters, and `argc`/`argv`.

5.1 Main Path

- Start from the question: What moves during a function call?
- Build the model: C passes values; changing a caller object requires passing an address to that object.
- End with a checkable action: Students can explain stack frames, prototypes, array parameters, and `argc`/`argv`.
- Keep only this slide on the horizontal path during the live lecture.



5.2 Worked Example

- Use this as the live trace: Compare `swap(a, b)` with `swap(&a, &b)` and make students predict the printout.
- Representative source slide: 280
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

5.3 Anecdote Or Motivation

Pass-by-value is simple and strict; pointers are how C asks for explicit sharing.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

5.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Example

```c
#include<stdio.h>
int main(int argc, char* argv[])
{
    int i;
    for (i=1;i<argc;i++)
    {
        printf("l'argument n°%d vaut %s ",i,argv[i]);
    }
}
```

What does prints execution of the command: ./prog foo 12 bar ?

#280

5.5 Backup Index

Original slides 279-280

The main function

An executable can be called with a list of arguments
./prog arg1 arg2 ... argn
These arguments are the parameters of the function main
The prototype of main function:
int main(int argc, char * argv[])
argc: size of argv (number of passed arguments + 1)
argv: is a table of strings
argv[0] is a char * pointing to the name of the program
argv[1] is a char * pointing to the 1st argument
argv[arc-1] is a char * pointing to the last argument
Remark
If we want to use an argument as int, we have to convert it!
int atoi(char * ch) of stdlib.h

#279

slide 279

slide 280

Anchors: The main function; Example

5.6 Backup: The main function

Original slide 279

```
An executable can be called with a list of arguments
./prog arg1 arg2 ... argn
These arguments are the parameters of the function main
The prototype of main function:
int main(int argc, char* argv[])
argc: size of argv (number of passed arguments + 1)
argv: is a table of strings
argv[0] is a char* pointing to the name of the program
argv[1] is a char* pointing to the 1st argument
argv[arc-1] is a char* pointing to the last argument
Remark
If we want to use an argument as int, we have to convert it!
int atoi(char* ch) of stdlib.h
```

The main function

An executable can be called with a list of arguments
./prog arg1 arg2 ... argn
These arguments are the parameters of the function main
The prototype of main function:
int main(int argc, char * argv[])
argc: size of argv (number of passed arguments + 1)
argv: is a table of strings
argv[0] is a char * pointing to the name of the program
argv[1] is a char * pointing to the 1st argument
argv[arc-1] is a char * pointing to the last argument
Remark
If we want to use an argument as int, we have to convert it!
int atoi(char * ch) of stdlib.h

#279

5.7 Backup: Example

Original slide 280

What does prints execution of the command: ./prog foo 12 bar ?

Example

```
#include<stdio.h>
int main(int argc, char* argv[])
{
   int i;
   for (i=1;i<argc;i++)
   {
       printf("l'argument n°%d vaut %s ",i,argv[i]);
   }
}
```

What does prints execution of the command: ./prog foo 12 bar ?

#280